Module 2

Systems of Equations

A *system of n simultaneous equations in n unknowns* can be written in the general form:

$$\begin{array}{rcl} f_1(x_1, x_2, x_3, \ldots, x_n) & = & 0 \\ f_2(x_1, x_2, x_3, \ldots, x_n) & = & 0 \\ f_3(x_1, x_2, x_3, \ldots, x_n) & = & 0 \\ & \vdots & \\ f_n(x_1, x_2, x_3, \ldots, x_n) & = & 0 \end{array} \tag{2.1}$$

or

$$f_i(\mathbf{x}) = 0 \qquad i = 1, 2, 3, \ldots, n$$

where $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$. The functions $f_1, f_2, f_3, \ldots, f_n$ may be *linear* in the unknowns $x_1, x_2, x_3, \ldots, x_n$, as in

$$\begin{array}{rcl} x_1 + 2x_2 + 4x_3 & = & 7 \\ 8x_1 + x_2 - 11x_3 & = & -2 \\ 7x_1 + 9x_2 - 8x_3 & = & 8 \end{array}$$

or *nonlinear*, as in

$$\begin{array}{rcl} {x_1}^2 - {x_2}^2 - {x_3}^2 & = & 0 \\ {x_1}^2 + 2\,x_1x_2 + x_2x_3 - 7 & = & 0 \\ 2\,x_1x_2 + {x_2}^2 - 5\,x_1x_3 + 2 & = & 0 \end{array}$$

and in

$$\begin{array}{rcl} 0.50\sin(x_1x_2) - 0.50x_1 - 0.25\dfrac{x_2}{\pi} & = & 0 \\ \left(1.00 - \dfrac{0.25}{\pi}\right)(e^{2x_1} - e) - 2.00e\,x_1 + e\dfrac{x_2}{\pi} & = & 0 \end{array}$$

2.1 Systems of Linear Equations

If the equations $f_i(\mathbf{x}) = 0$, $i = 1, 2, 3, \ldots, n$ are linear in the unknowns $x_1, x_2, x_3, \ldots, x_n$, then Eqs. (2.1) may be written in the form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + \cdots + a_{3n}x_n &= b_3 \\ \vdots \qquad\qquad & \vdots \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n &= b_n \end{aligned} \tag{2.2}$$

or, in matrix form,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & & & & \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} \tag{2.2}$$

or, more compactly,

$$A\mathbf{x} = \mathbf{b} \tag{2.2}$$

where
$A =$ matrix of coefficients
$\mathbf{b} =$ vector of constants (or right-hand side (RHS) vector)
$\mathbf{x} =$ vector of unknowns

Given A and $\mathbf{b}$, we want to find the vector $\mathbf{x}$ which satisfies Eqs. (2.2); we denote this particular $\mathbf{x}$ as $\bar{\mathbf{x}}$ (the solution vector).

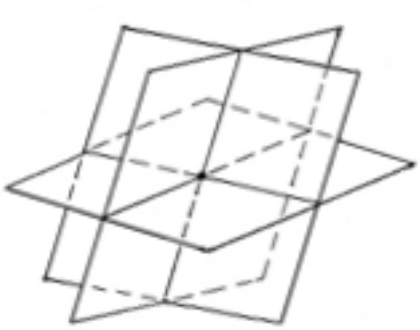
For any n by n matrix A, the following statements are equivalent, i.e., there are either all true or all false:

1. A is *nonsingular* (or *invertible*).
2. $A\mathbf{x} = \mathbf{b}$ has a unique solution for every $\mathbf{b}$ of size n.
3. $A\mathbf{x} = \mathbf{0}$ has only the trivial solution $\bar{\mathbf{x}} = \mathbf{0}$ (the zero vector).
4. The determinant of A is nonzero.
5. The row vectors of A are linearly independent.
6. The column vectors of A are linearly independent.

If A is *singular*, then Eqs. (2.2) either have *no solution* or *infinitely many solutions*. A system of linear equations with at least one solution is said to be *consistent* and one with no solution is said to be *inconsistent*.

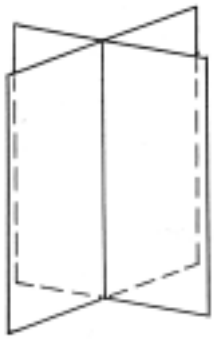
The figures below depict geometrically these three cases for a system of three equations in three unknowns, where each equation represents a 2-D plane in 3-D space. These constructions can be extended to the general case of n equations in n unknowns, where each equation now represents an $(n-1)$-D hyperplane in n-D hyperspace. It is of course rather difficult to visualize hyperplanes, let alone draw them on paper.

(a) Unique solution (consistent system; A nonsingular)

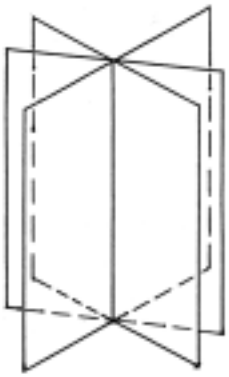


(b) Infinitely many solutions (consistent system; A singular)

3 coincident planes

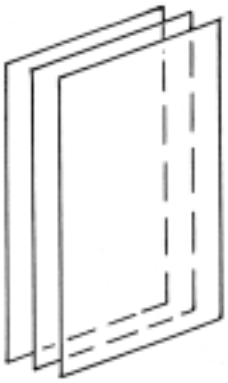


2 coincident planes intersecting a third plane

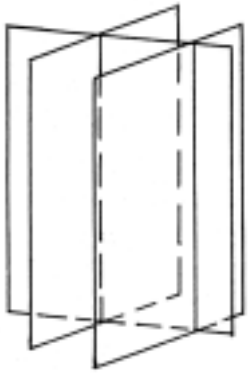


3 planes intersecting in a line

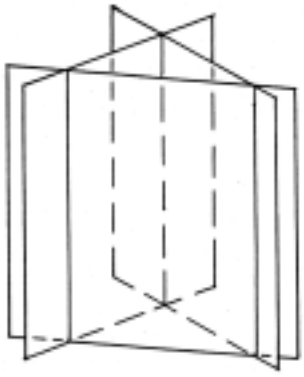
(c) No solution (inconsistent system; A singular)



3 parallel planes



a plane intersecting 2 parallel planes



3 planes with no common intersection

2.2 Methods for Solving the System $A\mathbf{x} = \mathbf{b}$

2.2.1 Direct Methods

- Yield the *exact* solution $\bar{\mathbf{x}}$ in a *finite* number of steps (assuming no round-off errors)
- Typically used when the linear system $A\mathbf{x} = \mathbf{b}$ is not too large (say, less than a hundred equations); usually, for such systems, the coefficient matrix A is *dense* (i.e., most of the a_{ij} are nonzero).
- When implemented on a computer, the entire coefficient matrix A is usually stored in main memory. The coefficients a_{ij} are modified during the computations.

- Use one of two approaches:
  1. elimination — the coefficient matrix A is transformed into an upper triangular matrix or into the identity matrix, with the vector of constants $\mathbf{b}$ correspondingly modified
  2. factorization — the coefficient matrix A is expressed as the product of two matrices, $A = LU$, where L is a lower triangular matrix and U is an upper triangular matrix
- Examples: Gaussian elimination method, Gauss-Jordan reduction method, Choleski's method, Crout's method, etc.

2.2.2 Indirect or Iterative Methods

- Generate a sequence of approximations, which may or may not converge to the solution. In principle, it takes an *infinite* number of iterations to obtain the exact solution $\bar{\mathbf{x}}$ (assuming no round-off errors); in practice, the iterations are terminated once the kth oteration estimate $\mathbf{x}_k$ satisfies a prescribed error criterion.
- Typically used when the linear system $A\mathbf{x} = \mathbf{b}$ is large (say, hundreds or thousands of equations); usually, for such systems, the coefficient matrix A is *sparse* (i.e., most of the a_{ij} are zero).
- When implemented on a computer, only the nonzero a_{ij}s (or blocks of these, with a few zero coefficients included) are stored in main memory; 'bookkeeping' techniques are utilized to determine the appropriate row an column indices. The coefficients a_{ij} are *not* modified during the computations.
- Examples: Jacobi method and Gauss-Seidel method (and variants of these), method of Kaczmarz, etc.

2.3 Elementary Transformation of Matrices

The direct methods for solving the linear system $A\mathbf{x} = \mathbf{b}$ are based on operations called *elementary transformations*, which are carried out with the use of *elementary matrices* of the *first*, *second* and *third kind*.

(a) An *elementary matrix of the first kind* is an $n \times n$ diagonal matrix, say Q, formed by taking the identity matrix and replacing the ith diagonal coefficient by a nonzero constant q.

Example:

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & q & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\det Q = q \qquad\qquad Q^{-1} = \mathrm{diag}(1, 1, 1/q, 1)$$

(b) An *elementary matrix of the second kind* is an $n \times n$ matrix, say R, formed by interchanging the ith and jth rows of the identity matrix.

Example:

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\det R = -1$$

(c) An *elementary matrix of the third kind* is an $n \times n$ diagonal matrix, say S, formed by taking the identity matrix and replacing the (i,j)th element, $i \neq j$, by a nonzero cosntant s. This can be viewed as multiplying the ith row of I by s and adding this to the jth row.

Example:

$$S = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ s & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$\det S = 1$$

Premultiplication of an arbitrary $n \times p$ matrix A by one fo these elementary matrices produces an *elementary transformation*, also called an *elementary row operation*.

Assume that

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix}.$$

Then:

$$QA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & q & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ qa_{31} & qa_{32} & qa_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix}.$$

Transformation: Third row of A is multiplied by scalar q.

$$RA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{31} & a_{32} & a_{33} \\ a_{21} & a_{22} & a_{23} \\ a_{41} & a_{42} & a_{43} \end{bmatrix}.$$

Transformation: Second and third rows of A are interchanged.

$$SA = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ s & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ sa_{11} + a_{31} & sa_{12} + a_{32} & sa_{13} + a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix}.$$

Transformation: The first row of A is multiplied by s and added to the third row.

Note that in each case, the operations performed on the identity matrix to obtain Q, R and S are the same operations produced on A by the application of Q, R and S on A. For example, to obtain S, we multiplied the first row of I by s and added this to the third row; note that this is precisely the transformation produced by S on A.

If A is a square matrix then:

$$\begin{aligned} \det(QA) = \det Q \times \det A &= q \det A \\ \det(RA) = \det R \times \det A &= -\det A \\ \det(SA) = \det S \times \det A &= \det A \end{aligned}$$

Postmultiplication of an arbitrary $p \times n$ matrix A by an elementary matrix produces an *elementary column operation*. This time, the transformations produced are:

AQ: the elements of one column are multiplied by q

AR: two columns are interchanged

AS: the elements of one column are multiplied by s and added to another column

2.4 Gaussian Elimination Method

The Gaussian elimination method, or GEM, consists of two passes, namely:

1. *forward elimination* — reduction of the coefficient matrix A to an upper triangular matrix, say $\acute{A}$, such that $\acute{a}_{ij} = 0 \; \forall \; j < i$, with concurrent modification of the vector of constants $\mathbf{b}$ to $\acute{\mathbf{b}}$, using elementary operations, to wit:
   (a) Any equation in the set can be multiplied (or divided) by a nonzero scalar without affecting the solution.
   (b) Any two equations in the set can be interchanged without affecting the solution.
   (c) Any equation in the set can be added to (or subtracted from) another equation without affecting the solution.

$$\left[\begin{array}{cccccc|c} a_{11} & a_{12} & a_{13} & a_{14} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & a_{23} & a_{24} & \cdots & a_{2n} & b_2 \\ a_{31} & a_{32} & a_{33} & a_{34} & \cdots & a_{3n} & b_3 \\ a_{41} & a_{42} & a_{43} & a_{44} & \cdots & a_{4n} & b_4 \\ & & & \vdots & & & \\ a_{n1} & a_{n2} & a_{n3} & a_{n4} & \cdots & a_{nn} & b_n \end{array}\right] \Longrightarrow \left[\begin{array}{cccccc|c} \acute{a}_{11} & \acute{a}_{12} & \acute{a}_{13} & \acute{a}_{14} & \cdots & \acute{a}_{1n} & \acute{b}_1 \\ 0 & \acute{a}_{22} & \acute{a}_{23} & \acute{a}_{24} & \cdots & \acute{a}_{2n} & \acute{b}_2 \\ 0 & 0 & \acute{a}_{33} & \acute{a}_{34} & \cdots & \acute{a}_{3n} & \acute{b}_3 \\ 0 & 0 & 0 & \acute{a}_{44} & \cdots & \acute{a}_{4n} & \acute{b}_4 \\ & & & \vdots & & & \\ 0 & 0 & 0 & 0 & \cdots & \acute{a}_{nn} & \acute{b}_n \end{array}\right]$$

2. *backward substitution* — solving for the unknowns x_i, $i = n, n-1, n-2, \ldots, 2, 1$.

$$\begin{aligned} x_n &\leftarrow \frac{\acute{b}_n}{\acute{a}_{nn}} \\ x_i &\leftarrow \frac{\acute{b}_i - \sum_{j=i+1}^{n} \acute{a}_{ij} x_j}{\acute{a}_{ii}} \qquad i = n-1, n-2, \ldots, 2, 1 \end{aligned}$$

2.4.1 Algorithm: GEM with Partial Pivoting by Row Interchange

Let k point to the unknown to be eliminated
i point to the equation (or row) from which x_k is to be eliminated
j point to the element (or column) to be operated on

Forward Elimination: Repeat for $k = 1, 2, \ldots, n-1$.

(a) Search for pivot element

$$\begin{aligned} \text{pivot element} &= \max(|a_{kk}|, |a_{(k+1)k}|, \ldots, |a_{nk}|) \\ &\equiv a_{rk} \equiv a_{\max} \end{aligned}$$

(b) Test for singularity

if $|a_{\max}| < \varepsilon$ then terminate computations

(c) Interchange the kth and rth equations to place $a_{\max}$ in the diagonal, or pivot, position

$$\begin{aligned} a_{kj} &\leftrightarrow a_{rj} \qquad j = k, k+1, \ldots, n \\ b_k &\leftrightarrow b_r \end{aligned}$$

(d) Normalize the kth, or pivot, row

$$\begin{aligned} \acute{a}_{kj} &\leftarrow \frac{a_{kj}}{a_{\max}} \qquad j = k, k+1, \ldots, n \\ \acute{b}_k &\leftarrow \frac{b_k}{a_{\max}} \end{aligned}$$

(e) Eliminate x_k from the $(k+1)$th to the nth equations

$$\left.\begin{array}{l} \acute{a}_{ij} \leftarrow a_{ij} - a_{ik}a_{kj} \qquad j = k, k+1, \ldots, n \\ \acute{b}_i \leftarrow b_i - a_{ik}b_k \end{array}\right\} i = k+1, \ldots, n$$

Backward Substitution

$$\begin{aligned} \bar{x}_n &\leftarrow b_n \\ \bar{x}_i &\leftarrow b_i - \sum_{j=i+1}^{n} a_{ij}\bar{x}_j \qquad i = n-1, n-2, \ldots, 2, 1 \end{aligned}$$

Example:

$$\begin{bmatrix} 2 & -7 & 4 \\ 1 & 9 & -6 \\ -3 & 8 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 1 \\ 6 \end{bmatrix}$$

FORWARD ELIMINATION

First stage

$$\left[\begin{array}{ccc|c} 2 & -7 & 4 & 9 \\ 1 & 9 & -6 & 1 \\ -3 & 8 & 5 & 6 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{ccc|c} -3 & 8 & 5 & 6 \\ 1 & 9 & -6 & 1 \\ 2 & -7 & 4 & 9 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 1 & 9 & -6 & 1 \\ 2 & -7 & 4 & 9 \end{array}\right] \overset{\text{E}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right]$$

Second stage

$$\left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{E}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right]$$

Final step: Normalize last row

$$\left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right] \overset{\text{T}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|c} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & 1 & 2 \end{array}\right]$$

BACKWARD SUBSTITUTION

$$\begin{aligned} \bar{x}_3 &= 2 \\ \bar{x}_2 &= \frac{9}{35} + \frac{13}{35}(2) = 1 \\ \bar{x}_1 &= -2 + \frac{8}{3}(1) + \frac{5}{3}(2) = 4 \end{aligned}$$

Hence:

$$\bar{\mathbf{x}} = \begin{bmatrix} 4 \\ 1 \\ 2 \end{bmatrix}$$

2.4.2 EASY Implementation

```
procedure GEM(A, n, ε, x, det)
  array A(1:n,1:n + 1), x(1:n)
  det ← 1.0
//FORWARD ELIMINATION//
  for k ← 1 to n − 1 do
//...Search for pivot element//
    amax ← A(k, k)
    r ← k
    for i ← k + 1 to n do
      if |A(i, k)| > |amax| then [amax ← A(i, k); r ← i]
    end for
//...Test for singularity//
    if |amax| < ε then [det ← 0; return]
                  else det ← det ∗ amax
//...Interchange rows, if necessary//
    if r ≠ k then [for j ← k to n + 1 do; A(k, j) ↔ A(r, j); end for; det ← −det]
//...Normalize the kth, or pivot, row//
    for j ← k to n + 1 do
      A(k, j) ← A(k, j)/amax
    end for
//...Eliminate x_k from the (k + 1)th to the nth rows//
    for i ← k + 1 to n do
      for j ← k to n + 1 do
        A(i, j) ← A(i, j) − A(i, k) ∗ A(k, j)
      end for
    end for
  end for                                              //forward elimination//
//...Normalize the nth row//
  if |A(n, n)| < ε then [det ← 0; return]
                else [A(n + 1, n) ← A(n + 1, n)/A(n, n); det ← det ∗ A(n, n); A(n, n) ← 1]
//BACKWARD SUBSTITUTION//
  x(n) ← A(n, n + 1)
  for i ← n − 1 to 1 by -1 do
    sum ← 0
    for j ← i + 1 to n do
      sum ← sum + A(i, j) ∗ x(j)
    end for
    x(i) ← A(i, n + 1) − sum
  end for                                              //backward substitution//
end procedure
```

Some notes on procedure GEM:

1. The vector of constants **b** is stored as the $(n+1)$th column of A to obtain more compact mode.
2. Actual interchange of elements may be avoided by using a permutation vector, say **p**, of size n and using $p(i)$, instead of i, as row subscript.
3. More nearly accurate results can be obtained by searching columns k thru n (rather than column k only) for the pivot element and then performing both row and column interchanges, as needed (this is called *total pivoting*).
4. The time complexity of GEM as implemented is $O(n^3)$. (Verify.)

5. A sample calling sequence for procedure GEM is:

```
input n, A, ε
call GEM(A, n, ε, x, det)
if det = 0 then [...]                    //x contains 'garbage'//
           else [...]                    //x contains solution vector//
```

2.5 Gauss-Jordan Reduction Method

The Gauss-Jordan reduction method, or GJRM, consists of one pass only (not two as in GEM) in which the coefficient matrix A is reduced to the identity matrix I, with the vector of constants $\mathbf{b}$ accordingly modified.

$$A\mathbf{x} = \mathbf{b} \Longrightarrow I\mathbf{x} = \acute{\mathbf{b}} \qquad \text{therefore } \bar{\mathbf{x}} = \acute{\mathbf{b}}$$

2.5.1 Algorithm

Repeat for $k = 1, 2, \ldots, n$:

(a) Search for pivot element

$$\begin{aligned} \text{pivot element} &= \max(|a_{kk}|, |a_{(k+1)k}|, \ldots, |a_{nk}|) \\ &\equiv a_{rk} \equiv a_{\max} \end{aligned}$$

(b) Test for singularity

if $|a_{\max}| < \varepsilon$ then terminate computations

(c) Interchange the kth and rth rows to place $a_{\max}$ in the diagonal, or pivot, position

$$\begin{aligned} a_{kj} &\leftrightarrow a_{rj} \qquad j = k, k+1, \ldots, n \\ b_k &\leftrightarrow b_r \end{aligned}$$

(d) Normalize the kth, or pivot, row

$$\begin{aligned} \acute{a}_{kj} &\leftarrow \frac{a_{kj}}{a_{\max}} \qquad j = k, k+1, \ldots, n \\ \acute{b}_k &\leftarrow \frac{b_k}{a_{\max}} \end{aligned}$$

(e) Eliminate x_k from *all* rows (equations) except the kth

$$\left.\begin{aligned} \acute{a}_{ij} &\leftarrow a_{ij} - a_{ik}a_{kj} \qquad j = k, k+1, \ldots, n \\ \acute{b}_i &\leftarrow b_i - a_{ik}b_k \end{aligned}\right\} i = 1, 2, \ldots, n, i \neq k$$

When the algorithm terminates, the modified vector $\acute{\mathbf{b}}$ contains the solution vector $\bar{\mathbf{x}}$.

Example:

$$\begin{bmatrix} 2 & -7 & 4 \\ 1 & 9 & -6 \\ -3 & 8 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 1 \\ 6 \end{bmatrix}$$

First stage

$$\left[\begin{array}{rrr|r} 2 & -7 & 4 & 9 \\ 1 & 9 & -6 & 1 \\ -3 & 8 & 5 & 6 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{rrr|r} -3 & 8 & 5 & 6 \\ 1 & 9 & -6 & 1 \\ 2 & -7 & 4 & 9 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 1 & 9 & -6 & 1 \\ 2 & -7 & 4 & 9 \end{array}\right] \overset{\text{R}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right]$$

Second stage

$$\left[\begin{array}{rrr|r} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{rrr|r} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & \frac{35}{3} & -\frac{13}{3} & 3 \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & -\frac{8}{3} & -\frac{5}{3} & -2 \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & -\frac{5}{3} & \frac{22}{3} & 13 \end{array}\right] \overset{\text{E}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & 0 & -\frac{93}{35} & -\frac{46}{35} \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right]$$

Third stage

$$\left[\begin{array}{rrr|r} 1 & 0 & -\frac{93}{35} & -\frac{46}{35} \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right] \overset{\text{T}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & 0 & -\frac{93}{35} & -\frac{46}{35} \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & \frac{47}{7} & \frac{94}{7} \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & 0 & -\frac{93}{35} & -\frac{46}{35} \\ 0 & 1 & -\frac{13}{35} & \frac{9}{35} \\ 0 & 0 & 1 & 2 \end{array}\right] \overset{\text{R}}{\Rightarrow} \left[\begin{array}{rrr|r} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \end{array}\right]$$

Hence:

$$\bar{\mathbf{x}} = \begin{bmatrix} 4 \\ 1 \\ 2 \end{bmatrix}$$

2.5.2 Matrix Inversion Using GJRM

The inverse of a matrix can be found by applying the Gauss-Jordan reduction method as follows:

$$AA^{-1} = I \Longrightarrow IA^{-1} = \acute{I} \qquad \text{therefore } A^{-1} = \acute{I}$$

Note that this is equivalent to solving a set of n $n \times n$ linear systems with the RHS vector equal to the unit vector $\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n$, respectively.

2.5.3 Algorithm for In-Place Matrix Inversion Using the Gauss-Jordan Reduction Method

The algorithm consists of two 'passes'. In Pass 1, the columns of A^{-1} are generated and overstored in successive columns of A. Assume that at the kth stage of the reduction process the original rth row is used as the pivot row; then the rth column of A^{-1} is 'born' and is stored in the kth column of A (and 'matures' there into its final form). Thus at the end of Pass 1, the transformed matrix $\acute{A}$, which should have been the identity matrix I, contains instead the columns of A^{-1}, but not in the correct order. To determine the correct order of the *columns* of A^{-1}, the history of the *row* interchanges performed during the reduction process must be recorded, say in the vector $\mathbf{p}$. If as the kth stage of the reduction process the original rth row is used as the pivot row, this is recorded as $p(k) = r$. Thus, $p(k) = r$ means that the rth column of A^{-1} is in the kth column of $\acute{A}$.

In Pass 2, the columns of $\acute{A}$ are unscrambled to form A^{-1} using the permutation vector $\mathbf{p}$.

Pass 1: Apply Gauss-Jordan reduction to generate the transformed matrix $\acute{A}$. Repeat for $k = 1, 2, \ldots, n$:

(a) Search for pivot element

$$\begin{aligned} \text{pivot element} &= \max(|a_{kk}|, |a_{(k+1)k}|, \ldots, |a_{nk}|) \\ &\equiv a_{rk} \equiv a_{\max} \end{aligned}$$

(b) Test for singularity

if $|a_{\max}| < \varepsilon$ then terminate computations

(c) Interchange the kth and rth rows to place $a_{\max}$ in the diagonal, or pivot, position

$$\begin{aligned} a_{kj} &\leftrightarrow a_{rj} \qquad j = 1, 2, \ldots, n \\ p_k &\leftrightarrow p_r \end{aligned}$$

(d) Normalize the kth, or pivot, row

$$\begin{aligned} \acute{a}_{kj} &\leftarrow \frac{a_{kj}}{a_{\max}} \qquad j = 1, 2, \ldots, n, j \neq k \\ \acute{a}_{kk} &\leftarrow \frac{1.0}{a_{\max}} \end{aligned}$$

(e) Reduce kth, or pivot, column to unit vector $\mathbf{e}_k$

$$\left.\begin{aligned} &\acute{a}_{ij} \leftarrow a_{ij} - a_{ik} a_{kj} \qquad j = 1, 2, \ldots, n, j \neq k \\ &\acute{a}_{ik} \leftarrow -\frac{a_{ik}}{a_{\max}} \end{aligned}\right\} i = 1, 2, \ldots, n, i \neq k$$

Pass 2: Unscramble the columns of $\acute{A}$ to form A^{-1}. Repeat for $i = 1, 2, \ldots, n$:

$$\begin{aligned} t_{p_j} &\leftarrow a_{ij} \qquad j = 1, 2, \ldots, n \\ a_{ij} &\leftarrow t_j \qquad j = 1, 2, \ldots, n \end{aligned}$$

Example: Matrix inversion using the GJRM

$$A = \begin{bmatrix} 1 & -2 & 0 \\ -2 & 1 & -2 \\ 0 & -2 & 1 \end{bmatrix}$$

First stage

$$\left[\begin{array}{ccc|ccc} 1 & -2 & 0 & 1 & 0 & 0 \\ -2 & 1 & -2 & 0 & 1 & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{ccc|ccc} -2 & 1 & -2 & 0 & 1 & 0 \\ 1 & -2 & 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & -\frac{1}{2} & 1 & 0 & -\frac{1}{2} & 0 \\ 1 & -2 & 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \end{array}\right] \overset{\text{R}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & -\frac{1}{2} & 1 & 0 & -\frac{1}{2} & 0 \\ 0 & -\frac{3}{2} & -1 & 1 & \frac{1}{2} & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \end{array}\right]$$

Second stage

$$\left[\begin{array}{ccc|ccc} 1 & -\frac{1}{2} & 1 & 0 & -\frac{1}{2} & 0 \\ 0 & -\frac{3}{2} & -1 & 1 & \frac{1}{2} & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \end{array}\right] \overset{\text{STI}}{\Longrightarrow} \left[\begin{array}{ccc|ccc} 1 & -\frac{1}{2} & 1 & 0 & -\frac{1}{2} & 0 \\ 0 & -2 & 1 & 0 & 0 & 1 \\ 0 & -\frac{3}{2} & -1 & 1 & \frac{1}{2} & 0 \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & -\frac{1}{2} & 1 & 0 & -\frac{1}{2} & 0 \\ 0 & 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & -\frac{3}{2} & -1 & 1 & \frac{1}{2} & 0 \end{array}\right] \overset{\text{R}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & 0 & \frac{3}{4} & 0 & -\frac{1}{2} & -\frac{1}{4} \\ 0 & 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{7}{4} & 1 & \frac{1}{2} & -\frac{3}{4} \end{array}\right]$$

Third stage

$$\left[\begin{array}{ccc|ccc} 1 & 0 & \frac{3}{4} & 0 & -\frac{1}{2} & -\frac{1}{4} \\ 0 & 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{7}{4} & 1 & \frac{1}{2} & -\frac{3}{4} \end{array}\right] \overset{\text{T}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & 0 & \frac{3}{4} & 0 & -\frac{1}{2} & -\frac{1}{4} \\ 0 & 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{7}{4} & 1 & \frac{1}{2} & -\frac{3}{4} \end{array}\right] \overset{\text{N}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & 0 & \frac{3}{4} & 0 & -\frac{1}{2} & -\frac{1}{4} \\ 0 & 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & 1 & -\frac{4}{7} & -\frac{2}{7} & \frac{3}{7} \end{array}\right] \overset{\text{R}}{\Rightarrow} \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{3}{7} & -\frac{2}{7} & -\frac{4}{7} \\ 0 & 1 & 0 & -\frac{2}{7} & -\frac{1}{7} & -\frac{2}{7} \\ 0 & 0 & 1 & -\frac{4}{7} & -\frac{2}{7} & \frac{3}{7} \end{array}\right]$$

Hence:

$$A^{-1} = \begin{bmatrix} \frac{3}{7} & -\frac{2}{7} & -\frac{4}{7} \\ -\frac{2}{7} & -\frac{1}{7} & -\frac{2}{7} \\ -\frac{4}{7} & -\frac{2}{7} & \frac{3}{7} \end{bmatrix}$$

Example: Matrix inversion using the GJRM (in-place version)

$$A = \begin{bmatrix} 1 & -2 & 0 \\ -2 & 1 & -2 \\ 0 & -2 & 1 \end{bmatrix} \qquad \begin{aligned} p_1 &= 1 \\ p_2 &= 2 \\ p_3 &= 3 \end{aligned}$$

First stage

$$\begin{bmatrix} 1 & -2 & 0 \\ -2 & 1 & -2 \\ 0 & -2 & 1 \end{bmatrix} \overset{\text{STI}}{\Longrightarrow} \begin{bmatrix} -2 & 1 & -2 \\ 1 & -2 & 0 \\ 0 & -2 & 1 \end{bmatrix} \overset{\text{N}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ 1 & -2 & 0 \\ 0 & -2 & 1 \end{bmatrix} \overset{\text{R}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ \frac{1}{2} & -\frac{3}{2} & -1 \\ 0 & -2 & 1 \end{bmatrix} \qquad \begin{aligned} p_1 &= 2 \\ p_2 &= 1 \\ p_3 &= 3 \end{aligned}$$

Second stage

$$\begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ \frac{1}{2} & -\frac{3}{2} & -1 \\ 0 & -2 & 1 \end{bmatrix} \overset{\text{STI}}{\Longrightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ 0 & -2 & 1 \\ \frac{1}{2} & -\frac{3}{2} & -1 \end{bmatrix} \overset{\text{N}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{3}{2} & -1 \end{bmatrix} \overset{\text{R}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{4} & \frac{3}{4} \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{3}{4} & -\frac{7}{4} \end{bmatrix} \qquad \begin{aligned} p_1 &= 2 \\ p_2 &= 3 \\ p_3 &= 1 \end{aligned}$$

Third stage

$$\begin{bmatrix} -\frac{1}{2} & -\frac{1}{4} & \frac{3}{4} \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{3}{4} & -\frac{7}{4} \end{bmatrix} \overset{\text{T}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{4} & \frac{3}{4} \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{3}{4} & -\frac{7}{4} \end{bmatrix} \overset{\text{N}}{\Rightarrow} \begin{bmatrix} -\frac{1}{2} & -\frac{1}{4} & \frac{3}{4} \\ 0 & -\frac{1}{2} & -\frac{1}{2} \\ -\frac{2}{7} & -\frac{3}{7} & -\frac{4}{7} \end{bmatrix} \overset{\text{R}}{\Rightarrow} \underbrace{\begin{bmatrix} -\frac{2}{7} & -\frac{4}{7} & \frac{3}{7} \\ -\frac{1}{7} & -\frac{2}{7} & -\frac{2}{7} \\ -\frac{2}{7} & -\frac{3}{7} & -\frac{4}{7} \end{bmatrix}}_{\acute{A}} \qquad \begin{aligned} p_1 &= 2 \\ p_2 &= 3 \\ p_3 &= 1 \end{aligned}$$

Using permutation vector $\mathbf{p}$, unscramble columns of $\acute{A}$ to form A^{-1}:

$p_1 = 2$ means column 2 of A^{-1} is in column 1 of $\acute{A}$
$p_2 = 3$ means column 3 of A^{-1} is in column 2 of $\acute{A}$
$p_3 = 1$ means column 1 of A^{-1} is in column 3 of $\acute{A}$

Hence:

$$A^{-1} = \begin{bmatrix} \frac{3}{7} & -\frac{2}{7} & -\frac{4}{7} \\ -\frac{2}{7} & -\frac{1}{7} & -\frac{2}{7} \\ -\frac{4}{7} & -\frac{2}{7} & \frac{3}{7} \end{bmatrix}$$

2.5.4 EASY Implementation

procedure GJRM(A, n, ε, $mode$, det)
//Solves the linear system $A\mathbf{x} = \mathbf{b}$. If $mode = -1$, only $\bar{\mathbf{x}}$ is determined; if $mode = 0$, only A^{-1} is determined; if $mode = +1$, both $\bar{\mathbf{x}}$ and A^{-1} are determined. The vector of constants $\mathbf{b}$ is stored as the $(n+1)$th column of A, and is replaced by $\bar{\mathbf{x}}$ if $mode = -1$ or $+1$. If A is singular, det is set to zero; else det contains $\det A$. A^{-1} replaces A if $mode = 0$ or $+1$.//
  array A(1:n,1:$n+1$), p(1:n), t(1:n)
//Initializations//
  $det \leftarrow 1.0$
  $m \leftarrow n + |mode|$

```
  for i ← 1 to n do
    p(i) ← i
  end for
//Perform Gauss-Jordan reduction//
  for k ← 1 to n do
//...Search for pivot element//
    amax ← A(k, k)
    r ← k
    for i ← k to n do
      if |A(i, k)| > |amax| then [amax ← A(i, k); r ← i]
    end for
//...Test for singularity//
    if |amax| < ε then [det ← 0; return]
                  else det ← det * amax
//...Interchange rows, if necessary//
    if r ≠ k then [for j ← 1 to m do; A(k, j) ↔ A(r, j); end for; p(k) ↔ p(r); det ← −det]
//...Normalize the kth, or pivot, row//
    for j ← 1 to m do
      A(k, j) ← A(k, j)/amax
    end for
    A(k, k) ← 1.0/amax
//...Reduce pivot column to unit vector e_k//
    for i ← 1 to n do
      if i ≠ k then [aik ← A(i, k); for j ← 1 to m do; A(i, j) ← A(i, j) − aik * A(k, j); end for; A(i, k) ←
−aik/amax]
    end for
  end for                                                    //Gauss-Jordan reduction//
  if mode = −1 then return                                   //only x̄ is desired//
//Unscramble the columns of Á to form A^−1//
  for i ← 1 to n do
    for j ← 1 to n do
      t(p(j)) ← A(i, j)
    end for
    for j ← 1 to n do
      A(i, j) ← t(j)
    end for
  end for                                                    //Unscramble ...//
end procedure
```

Some notes on procedure GJRM:

1. The time complexity of GJRM as implemented is $O(n^3)$, which is the same as GEM. However, the constant for GJRM is larger than that for GEM. (Verify.) If it is desired to find $\bar{\mathbf{x}}$ only, use GEM.

2. A sample calling sequence for procedure GJRM is:

```
input n, A
call GJRM(A, n, 10^−7, 1, det)
if det = 0 then [...]                                  //A contains 'garbage'//
           else [...]                                  //A contains A^−1 and x̄//
```

2.6 Matrix Inversion by Partitioning

Let

$$Q = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \qquad Q^{-1} = \begin{bmatrix} W & X \\ Y & Z \end{bmatrix},$$

where A, D, W and Z are square matrices. Then

$$QQ^{-1} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} W & X \\ Y & Z \end{bmatrix} = \begin{bmatrix} I & \mathbf{0} \\ \mathbf{0} & I \end{bmatrix},$$

or, multiplying out:

$$\begin{aligned} AW + BY &= I && (2.3)\\ AX + BZ &= \mathbf{0} && (2.4)\\ CW + DY &= \mathbf{0} && (2.5)\\ CX + DZ &= I && (2.6) \end{aligned}$$

Solving (2.3) and (2.5) simultaneously, we obtain:

$$\begin{aligned} W &= (A - BD^{-1}C)^{-1} && (2.7)\\ Y &= -D^{-1}CW = -D^{-1}C(A - BD^{-1}C)^{-1} && (2.8) \end{aligned}$$

Solving (2.4) and (2.6) simultaneously, we obtain:

$$\begin{aligned} Z &= (D - CA^{-1}B)^{-1} && (2.9)\\ X &= -A^{-1}BZ = -A^{-1}B(D - CA^{-1}B)^{-1} && (2.10) \end{aligned}$$

Eqs. (2.7) through (2.10) are particularly useful if the partitioning is such that $B = C = \mathbf{0}$; then, $X = Y = \mathbf{0}$, $W = A^{-1}$ and $Z = D^{-1}$. Hence:

$$Q^{-1} = \begin{bmatrix} A^{-1} & \mathbf{0} \\ \mathbf{0} & D^{-1} \end{bmatrix}.$$

Example:

$$Q = \begin{bmatrix} 3 & 2 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix}$$

Let Q be partitioned such that

$$Q = \left[\begin{array}{ccc|cc} 3 & 2 & 0 & 0 & 0 \\ 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ \hline 0 & 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 1 & 2 \end{array}\right] = \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} A & \mathbf{0} \\ \mathbf{0} & D \end{bmatrix}.$$

Then,

$$Q^{-1} = \begin{bmatrix} A^{-1} & \mathbf{0} \\ \mathbf{0} & D^{-1} \end{bmatrix}.$$

To find A^{-1}, partition A such that

$$A = \left[\begin{array}{cc|c} 3 & 2 & 0 \\ 1 & 2 & 0 \\ \hline 0 & 0 & 2 \end{array}\right] = \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} E & \mathbf{0} \\ \mathbf{0} & H \end{bmatrix}.$$

Then,

$$A^{-1} = \begin{bmatrix} E^{-1} & \mathbf{0} \\ \mathbf{0} & H^{-1} \end{bmatrix},$$

where

$$\begin{aligned} E^{-1} &= \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}^{-1} = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{4} & \frac{3}{4} \end{bmatrix} = D^{-1} \text{ since } D = E \\ H^{-1} &= \frac{1}{2} \end{aligned}$$

Hence

$$Q = \left[\begin{array}{cc|c|cc} \frac{1}{2} & -\frac{1}{2} & 0 & 0 & 0 \\ -\frac{1}{4} & \frac{3}{4} & 0 & 0 & 0 \\ \hline 0 & 0 & \frac{1}{2} & 0 & 0 \\ \hline 0 & 0 & 0 & \frac{1}{2} & -\frac{1}{2} \\ 0 & 0 & 0 & -\frac{1}{4} & \frac{3}{4} \end{array}\right]$$

2.7 Inversion of Complex Matrices

Let

$$\begin{aligned} C &= A + iB \\ C^{-1} &= X + iY \end{aligned}$$

where A, B, X and Y are real matrices. Then

$$(A + iB)(X + iY) = I + i\mathbf{0},$$

or, multiplying out

$$AX - BY = I \tag{2.11}$$
$$AY + BX = \mathbf{0} \tag{2.12}$$

If A^{-1} exists, then from (2.12):

$$AY = -BX,$$

or

$$Y = -A^{-1}BX.$$

Substituting in (2.11), we get

$$\begin{aligned} AX - B(-A^{-1}BX) &= I \\ (A + BA^{-1}B)X &= I \end{aligned}$$

or

$$X = (A + BA^{-1}B)^{-1}$$

and therefore

$$Y = -A^{-1}B(A + BA^{-1}B)^{-1}.$$

If B^{-1} exists, then from (2.11):

$$BX = -AY,$$

or

$$X = -B^{-1}AY.$$

Substituting in (2.12), we get

$$\begin{aligned} A(-B^{-1}AY) - BY &= I \\ -(AB^{-1}A + B)Y &= I \end{aligned}$$

or

$$Y = -(AB^{-1}A + B)^{-1}$$

and therefore

$$X = B^{-1}A(AB^{-1}A + B)^{-1}.$$

If both A and B are non-singular, the two sets of expressions for X and Y are equivalent. (Verify.)

Example:

$$
\begin{aligned}
C &= \begin{bmatrix} 3+2i & 2-i \\ 1-2i & 2+i \end{bmatrix} = \underbrace{\begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}}_{A} + \underbrace{\begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix}}_{B} i \\
C^{-1} &= X + Yi \qquad \text{where} \quad \begin{aligned} X &= (A + BA^{-1}B)^{-1} \\ Y &= -A^{-1}BX \end{aligned} \\
A^{-1} &= \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}^{-1} = \frac{1}{4}\begin{bmatrix} 2 & -2 \\ -1 & 3 \end{bmatrix} \\
A^{-1}B &= \frac{1}{4}\begin{bmatrix} 2 & -2 \\ -1 & 3 \end{bmatrix}\begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix} \\
BA^{-1}B &= \begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix}\begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} 6 & -3 \\ -6 & 3 \end{bmatrix} \\
A + BA^{-1}B &= \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 6 & -3 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 9 & -1 \\ -5 & 5 \end{bmatrix} \\
X = (A + BA^{-1}B)^{-1} &= \begin{bmatrix} 9 & -1 \\ -5 & 5 \end{bmatrix}^{-1} = \frac{1}{40}\begin{bmatrix} 5 & 1 \\ 5 & 9 \end{bmatrix} \\
Y = A^{-1}BX &= \begin{bmatrix} 2 & -1 \\ -2 & 1 \end{bmatrix}\left\{\frac{1}{40}\begin{bmatrix} 5 & 1 \\ 5 & 9 \end{bmatrix}\right\} = \frac{1}{40}\begin{bmatrix} -5 & 7 \\ 5 & -7 \end{bmatrix}
\end{aligned}
$$

Hence:

$$
C^{-1} = X + Yi = \frac{1}{40}\begin{bmatrix} 5 & 1 \\ 5 & 9 \end{bmatrix} + \frac{1}{40}\begin{bmatrix} -5 & 7 \\ 5 & -7 \end{bmatrix} i = \frac{1}{40}\begin{bmatrix} 5-5i & 1+7i \\ 5+5i & 9-7i \end{bmatrix}.
$$

A 'complex' version of procedure GJRM given in the notes yields directly the same result.

$$
C^{-1} = \begin{bmatrix} 0.1250 - 0.1250i & 0.0250 + 0.1750i \\ 0.1250 + 0.1250i & 0.2250 - 0.1750i \end{bmatrix}
$$

2.8 LU Factorization Using GEM

In applications where the linear system $A\mathbf{x} = \mathbf{b}$ is solved repeatedly with the same coefficient matrix A but with different RHS vectors $\mathbf{b}$, it is more computationally efficient to factor A once and then solve the factored system for each new $\mathbf{b}$. We saw earlier that solving the linear system $A\mathbf{x} = \mathbf{b}$ using GEM takes $O(n^3)$ time. We will see shortly that factoring A into the product LU can be done in-place with time complexity $O(n^3)$ and that solving the factored system $LU\mathbf{x} = \mathbf{b}$ takes $O(n^2)$ time. Thus each repeated solve takes $O(n^2)$ time only.

The algorithm to solve the linear system $A\mathbf{x} = \mathbf{b}$ based on LU factorization of A using GEM consists of three 'passes'. In Pass 1, the matrix A is factored as the product LU where L is a unit lower triangular matrix and U is upper triangular.

$$
L = \begin{bmatrix} 1 & & & & \\ \ell_{21} & 1 & & \mathbf{0} & \\ \ell_{31} & \ell_{32} & 1 & & \\ \vdots & & & \ddots & \\ \ell_{n1} & \ell_{n2} & \ell_{n3} & \cdots & 1 \end{bmatrix} \qquad U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ & u_{22} & u_{23} & \cdots & u_{2n} \\ & & u_{33} & \cdots & u_{3n} \\ & \mathbf{0} & & \ddots & \vdots \\ & & & & u_{nn} \end{bmatrix}
$$

The matrix U is simply the resulting upper triangular matrix at the end of the forward elimination pass of the GEM, with the normalization step omitted at each stage of teh elimination (why?). The elements of L, on the other hand, are simply the multipliers used during the elimination, i.e. the a_{ik}/a_{kk}. At the end of Pass 1, the linear system $A\mathbf{x} = \mathbf{b}$ becomes $LU\mathbf{x} = \mathbf{b}$. Now, let $\mathbf{z}$ be $U\mathbf{x}$; then $L\mathbf{z} = \mathbf{b}$.

In Pass 2, the linear system $L\mathbf{z} = \mathbf{b}$ is solved for $\mathbf{z}$ by forward substitution. If, during the factorization of A in Pass 1, row interchanges are performed, then $\mathbf{b}$ must be replaced with $\acute{\mathbf{b}}$, where $\acute{\mathbf{b}}$ is simply $\mathbf{b}$ with its elements permuted accoring to the row interchanges performed on A. This means that we must keep a history of the row interchanges performed, say in the permutation vector $\mathbf{p}$.

Finally, in Pass 3, the linear system $U\mathbf{x} = \mathbf{z}$ is solved for $\mathbf{x}$ by backward substitution.

If the linear system $A\mathbf{x} = \mathbf{b}$ is to be solved repeatedly with the same A but different $\mathbf{b}$s, it is clear that only passes 2 and 3 of the algorithm need to be performed for each repeated solve.

Example:

$$\begin{bmatrix} 2 & -4 & -2 \\ 4 & -2 & 4 \\ -1 & 5 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 20 \\ -5 \end{bmatrix} \qquad \begin{aligned} p_1 &= 1 \\ p_2 &= 2 \\ p_3 &= 3 \end{aligned}$$

1. FACTORIZATION USING GEM: $A \Rightarrow LU$

   First stage

$$\begin{bmatrix} 2 & -4 & -2 \\ 4 & -2 & 4 \\ -1 & 5 & 2 \end{bmatrix} \Rightarrow \begin{bmatrix} 4 & -2 & 4 \\ 2 & -4 & -2 \\ -1 & 5 & 2 \end{bmatrix} \Rightarrow \begin{bmatrix} 4 & -2 & 4 \\ \frac{1}{2} & -3 & -4 \\ -\frac{1}{4} & \frac{9}{2} & 3 \end{bmatrix} \qquad \begin{aligned} p_1 &= 2 \\ p_2 &= 1 \\ p_3 &= 3 \end{aligned}$$

   Second stage

$$\begin{bmatrix} 4 & -2 & 4 \\ \frac{1}{2} & -3 & -4 \\ -\frac{1}{4} & \frac{9}{2} & 3 \end{bmatrix} \Rightarrow \begin{bmatrix} 4 & -2 & 4 \\ -\frac{1}{4} & \frac{9}{2} & 3 \\ \frac{1}{2} & -3 & -4 \end{bmatrix} \Rightarrow \begin{bmatrix} 4 & -2 & 4 \\ -\frac{1}{4} & \frac{9}{2} & 3 \\ \frac{1}{2} & -\frac{2}{3} & -2 \end{bmatrix} \qquad \begin{aligned} p_1 &= 2 \\ p_2 &= 3 \\ p_3 &= 1 \end{aligned}$$

   Hence:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \qquad U = \begin{bmatrix} 4 & -2 & 4 \\ 0 & \frac{9}{2} & 3 \\ 0 & 0 & -2 \end{bmatrix} \qquad \mathbf{p} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}$$

2. FORWARD SUBSTITUTION TO SOLVE $L\mathbf{z} = \mathbf{b}$

$$\begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{4} & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} 20 \\ -5 \\ 4 \end{bmatrix} \begin{aligned} &\Leftarrow b_{p_1} = b_2 = 20 \\ &\Leftarrow b_{p_2} = b_3 = -5 \\ &\Leftarrow b_{p_3} = b_1 = 4 \end{aligned}$$

$$\begin{aligned} z_1 &= 20 \\ -\tfrac{1}{4}(20) + z_2 = -5 &\rightsquigarrow z_2 = 0 \\ \tfrac{1}{2}(20) - \tfrac{2}{3}(0) + z_3 = 4 &\rightsquigarrow z_3 = -6 \end{aligned}$$

3. BACKWARD SUBSTITUTION TO SOLVE $U\mathbf{x} = \mathbf{z}$

$$\begin{bmatrix} 4 & -2 & 4 \\ 0 & \frac{9}{2} & 3 \\ 0 & 0 & -2 \end{bmatrix} \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \bar{x}_3 \end{bmatrix} = \begin{bmatrix} 20 \\ 0 \\ -6 \end{bmatrix}$$

$$\bar{\mathbf{x}} = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix} \qquad \text{(Verify)}$$

2.8.1 Algorithm: Solving $A\mathbf{x} = \mathbf{b}$ Using GEM with In-Place LU Factorization of A

1. LU factorization of A: Repeat for $k = 1, 2, \ldots, n-1$

   (a) Search for the pivot element

   $$\begin{aligned} \text{pivot element} &= \max(|a_{kk}|, |a_{(k+1)k}|, \ldots, |a_{nk}|) \\ &\equiv a_{rk} \equiv a_{\max} \end{aligned}$$

   (b) Test for singularity

   if $|a_{\max}| < \varepsilon$ then terminate computations

   (c) Interchange the kth and rth equations to place $a_{\max}$ in the diagonal, or pivot, position

   $$\begin{aligned} a_{kj} &\leftrightarrow a_{rj} \qquad j = 1, 2, \ldots, n \\ p_k &\leftrightarrow p_r \end{aligned}$$

   (d) Eliminate x_k from the $(k+1)$th to the nth equations, replacing annihilated coefficients with the corresponding multipliers

   $$\left.\begin{aligned} \acute{a}_{ik} &\leftarrow \frac{a_{ik}}{a_{kk}} \\ \acute{a}_{ij} &\leftarrow a_{ij} - \acute{a}_{ik} a_{kj} \qquad j = k+1, \ldots, n \end{aligned}\right\} i = k+1, \ldots, n$$

2. Forward substitution (Note: $\ell_{ij} \equiv a_{ij} \forall j < i$)

   $$\begin{aligned} z_1 &= b_{p_1} \\ z_i &= b_{p_i} - \sum_{j=1}^{i-1} a_{ij} z_j \qquad i = 2, 3, \ldots, n \end{aligned}$$

3. Backward substitution (Note: $u_{ij} \equiv a_{ij} \forall j \geq i$)

   $$\begin{aligned} \bar{x}_n &= \frac{z_n}{a_{nn}} \\ \bar{x}_i &= \frac{z_i - \sum_{j=i+1}^{n} a_{ij} \bar{x}_j}{a_{ii}} \qquad i = n-1, n-2, \ldots, 2, 1 \end{aligned}$$

2.8.2 EASY Implementation

```
procedure FGEM(A, n, ε, p, det)
//Generates the LU factorization of A using GEM with partial pivoting by row interchange. LU is
overstored in A. p contains the history of row interchanges performed on A.//
  array A(1:n,1:n), p(1:n)
//...Initializations//
  for i ← 1 to n do
    p(i) ← i
  end for
  det ← 1.0
//PASS 1: LU factorization of A//
  for k ← 1 to n − 1 do
//...Search for pivot element//
    amax ← A(k, k)
    r ← k
    for i ← k + 1 to n do
      if |A(i, k)| > |amax| then [amax ← A(i, k); r ← i]
```

```
      end for
//...Test for singularity//
      if |amax| < ε then [det ← 0; return]
                    else det ← det * amax
//...Interchange rows, if necessary//
      if r ≠ k then [for j ← 1 to n do; A(k,j) ↔ A(r,j); end for; p(k) ↔ p(r); det ← −det]
//...Eliminate x_k from the (k+1)th to the nth rows; replace annihilated coefficients with corresponding
multipliers//
      for i ← k + 1 to n do
         A(i,k) ← A(i,k)/A(k,k)
         for j ← k + 1 to n do
            A(i,j) ← A(i,j) − A(i,k) * A(k,j)
         end for
      end for
   end for                                                    //LU factorization ...//
//...Check last pivot element and return//
   if |A(n,n)| < ε then det ← 0
                    else det ← det * A(n,n)
   return
end procedure
```

```
procedure SOLVE(A, n, p, b, x)
//Solves the system LUx = b, where LU is overstored in A. p contains the history of the row inter-
changes performed during the LU factorization of A.//
   array A(1:n,1:n), p(1:n), b(1:n), z(1:n), x(1:n)
//PASS 2: Forward substitution to solve for z//
   z(1) ← b(p(1))
   for i ← 2 to n do
      sum ← 0
      for j ← 1 to i − 1 do
         sum ← sum + A(i,j) * z(j)
      end for
      z(i) ← b(p(i)) − sum
   end for
//PASS 3: Backward substitution to solve for x//
   x(n) ← z(n)/A(n,n)
   for i ← n − 1 to 1 by -1 do
      sum ← 0
      for j ← i + 1 to n do
         sum ← sum + A(i,j) * x(j)
      end for
      x(i) ← (z(i) − sum)/A(i,i)
   end for
end procedure
```

Some notes on procedures FGEM and SOLVE:

1. The time complexity of FGEM as implemented is $O(n^3)$ and that of SOLVE is $O(n^2)$. Verify.

2. A sample calling sequence for FGEM and SOLVE is:

```
input n, A, ε
output A, ε
call FGEM(A, n, ε, p, det)
if det = 0 then [output 'A is singular'; stop]
           else [repeat for each new b; input b; call SOLVE(A, n, p, b, x); output b, x]
```

2.9 Choleski Factorization

1. An $n \times n$ *symmetric* matrix A is said to be *positive definite* if $\mathbf{x}^{\mathrm{t}} A\mathbf{x} > 0$ for every nonzero n-dimensional column vector $\mathbf{x}$.

   Example: The matrix $A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$ is positive definite since

   $$\mathbf{x}^{\mathrm{t}} A\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = {x_1}^2 + (x_1 - x_2)^2 + (x_2 - x_3)^2 + {x_3}^2 > 0$$

   for all column vectors $\mathbf{x}$ of size 3, $\mathbf{x} \neq \mathbf{0}$.

2. Two important properties of a symmetric positive definite matrix, in the context of solving the linear system $A\mathbf{x} = \mathbf{b}$, are:
   (a) It is nonsingular.
   (b) It can be factored into the form $A = LU$ where L and U are lower and upper triangular matrices, respectively, and $U = L^{\mathrm{t}}$.

3. The factorization $A = LU$ is carried out using *Choleski's algorithm*. This is a direct factorization algorithm which requires no row interchanges.

4. As in the factorization version of GEM, solving the linear system $A\mathbf{x} = \mathbf{b}$ consists of three 'passes', namely:
   (a) Find the LU factorization of A using Choleski's algorithm; then $LU\mathbf{x} = \mathbf{b}$. Let $\mathbf{z} = U\mathbf{x}$; then $L\mathbf{z} = \mathbf{b}$.
   (b) Solve the linear system $L\mathbf{z} = \mathbf{b}$ for $\mathbf{z}$ by forward substitution.
   (c) Solve the linear system $U\mathbf{x} = \mathbf{z}$ for $\mathbf{x}$ by backward substitution.

2.9.1 Algorithm

Let

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ & \vdots & & & \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix} \qquad L = \begin{bmatrix} \ell_{11} & & & & \\ \ell_{21} & \ell_{22} & & \mathbf{0} & \\ \ell_{31} & \ell_{32} & \ell_{33} & & \\ \vdots & & & \ddots & \\ \ell_{n1} & \ell_{n2} & \ell_{n3} & \cdots & \ell_{nn} \end{bmatrix}$$

Then:

(a) Generate the first column of L

$$\begin{aligned} \ell_{11} &\leftarrow \sqrt{a_{11}} \\ \ell_{i1} &\leftarrow \frac{a_{i1}}{\ell_{11}} \qquad\qquad i = 2, 3, \ldots, n \end{aligned}$$

(b) Generate 2nd to $(n-1)$th columns of L from diagonal element down

$$\left.\begin{aligned} \ell_{jj} &\leftarrow \sqrt{a_{jj} - \sum_{k=1}^{j-1} {\ell_{jk}}^2} \\ \ell_{ij} &\leftarrow \frac{a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk}}{\ell_{jj}} \qquad i = j+1, \ldots, n \end{aligned}\right\} j = 2, \ldots, n-1$$

(c) Generate the last diagonal element of L

$$\ell_{nn} \leftarrow \sqrt{a_{nn} - \sum_{k=1}^{n-1} {\ell_{nk}}^2}$$

An examination of Choleski's algorithm shows that the matrix L can be overstored in the original matrix A. An element a_{ij} is used only once; subsequent references to the (i,j)th position are references to the generated value ℓ_{ij}. Likewise, U need not be explicitly generated since $u_{ij} = \ell_{ji}$.

Example:

$$\begin{bmatrix} 6 & 2 & 1 & -1 \\ 2 & 4 & 1 & 0 \\ 1 & 1 & 4 & -1 \\ -1 & 0 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ -4 \\ 3 \end{bmatrix}$$

1. FACTORIZATION BY CHOLESKI'S ALGORITHM: $A \Rightarrow LU$

$$\begin{aligned}
\ell_{11} &= \sqrt{a_{11}} = \sqrt{6} = 2.4495 \\
\ell_{21} &= \frac{a_{21}}{\ell_{11}} = \frac{2}{2.4495} = 0.8165 \\
\ell_{31} &= \frac{a_{31}}{\ell_{11}} = \frac{1}{2.4495} = 0.4082 \\
\ell_{41} &= \frac{a_{41}}{\ell_{11}} = \frac{-1}{2.4495} = -0.4082 \\
\ell_{22} &= \sqrt{a_{22} - {\ell_{21}}^2} = \sqrt{4 - (0.8165)^2} = 1.8257 \\
\ell_{32} &= \frac{a_{32} - \ell_{31}\ell_{21}}{\ell_{22}} = \frac{1 - (0.4082)(0.8165)}{1.8257} = 0.3652 \\
\ell_{42} &= \frac{a_{42} - \ell_{41}\ell_{21}}{\ell_{22}} = \frac{0 - (-0.4082)(0.8165)}{1.8257} = 0.1826 \\
\ell_{33} &= \sqrt{a_{33} - {\ell_{31}}^2 - {\ell_{32}}^2} = \sqrt{4 - (0.4082)^2 - (0.3652)^2} = 1.9235 \\
\ell_{43} &= \frac{a_{43} - \ell_{41}\ell_{31} - \ell_{42}\ell_{32}}{\ell_{33}} = \frac{-1 - (-0.4082)(0.4082) - (0.1826)(0.3652)}{1.9235} = -0.4679 \\
\ell_{44} &= \sqrt{a_{44} - {\ell_{41}}^2 - {\ell_{42}}^2 - {\ell_{43}}^2} = \sqrt{3 - (-0.4082)^2 - (0.1826)^2 - (-0.4679)^2} = 1.6066
\end{aligned}$$

Hence:

$$L = \begin{bmatrix} 2.4495 & 0 & 0 & 0 \\ 0.8165 & 1.8257 & 0 & 0 \\ 0.4082 & 0.3652 & 1.9235 & 0 \\ -0.4082 & 0.1826 & -0.4679 & 1.6066 \end{bmatrix}; \qquad U = L^{\mathrm{t}}$$

2. FORWARD SUBSTITUTION TO SOLVE $L\mathbf{z} = \mathbf{b}$

$$\begin{bmatrix} 2.4495 & 0 & 0 & 0 \\ 0.8165 & 1.8257 & 0 & 0 \\ 0.4082 & 0.3652 & 1.9235 & 0 \\ -0.4082 & 0.1826 & -0.4679 & 1.6066 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ -4 \\ 3 \end{bmatrix}$$

$$\begin{aligned}
z_1 &= \frac{4}{2.4495} = 1.6330 \\
z_2 &= \frac{1 - (0.8165)(1.6330)}{1.8257} = -0.1826 \\
z_3 &= \frac{-4 - (0.4082)(1.6330) - (0.3652)(-0.1826)}{1.9235} = -2.3914 \\
z_4 &= \frac{3 - (-0.4082)(1.6330) - (0.1826)(-0.1826) - (-0.4679)(-2.3914)}{1.6066} = 1.6065
\end{aligned}$$

3. BACKWARD SUBSTITUTION TO SOLVE $U\mathbf{x} = \mathbf{z}$

$$\begin{bmatrix} 2.4495 & 0.8165 & 0.4082 & -0.4082 \\ 0 & 1.8257 & 0.3652 & 0.1826 \\ 0 & 0 & 1.9235 & -0.4679 \\ 0 & 0 & 0 & 1.6066 \end{bmatrix} \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \bar{x}_3 \\ \bar{x}_4 \end{bmatrix} = \begin{bmatrix} 1.6330 \\ -0.1826 \\ -2.3194 \\ 1.6065 \end{bmatrix}$$

$$\begin{aligned} \bar{x}_4 &= \frac{1.6065}{1.6066} = 1.0000 \\ \bar{x}_3 &= \frac{-2.3914 - (-0.4679)(1.0000)}{1.9235} = -1.0000 \\ \bar{x}_2 &= \frac{-0.1826 - (0.3652)(-1.0000) - (0.1826)(1.0000)}{1.8257} = 0 \\ \bar{x}_1 &= \frac{1.6330 - (0.8165)(0) - (0.4082)(-1.0000) - (-0.4082)(1.0000)}{2.4495} = 1.0000 \end{aligned}$$

Hence:

$$\bar{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \\ -1 \\ 1 \end{bmatrix}.$$

2.9.2 EASY Implementation

```
procedure CHOLESKI(A, n)
//Performs an in-place LU factorization of A using Choleski's algorithm. A must be symmetric and
positive definite. Only the elements of L are generated and are overstored in the lower triangular posi-
tions of A.//
  array A(1:n,1:n)
//...Generate the first column of L//
  A(1,1) ← √A(1,1)
  for i ← 2 to n do
    A(i,1) ← A(i,1)/A(1,1)
  end for
//...Generate the remaining columns of L//
  for j ← 2 to n do
    sum ← 0
    for k ← 1 to j − 1 do
      sum ← sum + A(j,k) * A(j,k)
    end for
    A(j,j) ← √(A(j,j) − sum)
    if j = n then return
    for i ← j + 1 to n do
      sum ← 0
      for k ← 1 to j − 1 do
        sum ← sum + A(i,k) * A(j,k)
      end for
      A(i,j) ← (A(i,j) − sum)/A(j,j)
    end for
  end for                                          //Generate remaining ...//
end procedure

procedure SOLVE(A, n, b, x)
//Solves the system LUx = b, where LU is the in-place Choleski factorization of A.//
  array A(1:n,1:n), b(1:n), z(1:n), x(1:n)
//Forward substitution to solve the linear system Lz = b. Note: L(i,j) ≡ A(i,j)//
  z(1) ← b(1)/A(1,1)
```

```
  for i ← 2 to n do
    sum ← 0
    for j ← 1 to i − 1 do
      sum ← sum + A(i, j) ∗ z(j)
    end for
    z(i) ← (b(i) − sum)/A(i, i)
  end for
//Backward substitution to solve the linear system Ux = z. Note: U(i, j) ≡ A(j, i)//
  x(n) ← z(n)/A(n, n)
  for i ← n − 1 to 1 by -1 do
    sum ← 0
    for j ← i + 1 to n do
      sum ← sum + A(j, i) ∗ x(j)
    end for
    x(i) ← (z(i) − sum)/A(i, i)
  end for
end procedure
```

2.10 Measures of Closeness

When the linear system $A\mathbf{x} = \mathbf{b}$ is solved, the computed solution, say $\bar{\mathbf{x}}_{\mathrm{c}}$, will in all likelihood be an approximation only to the true solution, say $\bar{\mathbf{x}}_{\mathrm{t}}$. So we ask: How do we measure the 'goodness' of the computed solution?

Two common measures are the *error*

$$\mathbf{e} = \bar{\mathbf{x}}_{\mathrm{t}} - \bar{\mathbf{x}}_{\mathrm{c}}$$

and the *residual*

$$\mathbf{r} = \mathbf{b} - A\bar{\mathbf{x}}_{\mathrm{c}}$$

The error vector $\mathbf{e}$ is a measure of how close the computed solution is to the true solution; the residual vector $\mathbf{r}$ is a measure of how close the computed RHS is to the true RHS. In general, we cannot directly calculate $\mathbf{e}$, but we can readily calculate $\mathbf{r}$. If the linear system is *well-conditioned* (i.e. A is not nearly singular), the residual provides a good estimate of the error. However, if the system is *ill-conditioned* (i.e. A is nearly singular), a small residual is no guarantee that the computed solution is close to the true solution.

Example 1: Consider the linear system $A\mathbf{x} = \mathbf{b}$

$$\begin{aligned} 8.00002x_1 + 4x_2 + 4x_3 &= 20 \\ 6.00001x_1 + 6x_2 + 3x_3 &= 21 \\ 2.00002x_1 + 8x_2 + x_3 &= 19 \end{aligned} \tag{2.13}$$

Suppose, upon solving this system, we arrive at the computed solution $\bar{\mathbf{x}}_{\mathrm{c}} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$. Calculating the residual, we obtain

$$\mathbf{r} = \begin{bmatrix} 20 \\ 21 \\ 19 \end{bmatrix} - \begin{bmatrix} 8.00002 & 4 & 4 \\ 6.00001 & 6 & 3 \\ 2.00002 & 8 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.00002 \\ 0.00001 \\ 0.00001 \end{bmatrix}$$

Relative to the problem data and the computed solution, these residuals are unquestionably small, leading us to expect that the computed solution $\bar{\mathbf{x}}_{\mathrm{c}} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ must be very close to the true solution. In fact,

the exact solution to Eqs. (2.13) is $\bar{\mathbf{x}}_{\mathrm{t}} = \begin{bmatrix} 0 \\ 2 \\ 3 \end{bmatrix}$! What happened?

Actually, the vector $\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ is the exact solution to the linear system $A\mathbf{x} = \acute{\mathbf{b}}$

$$\begin{aligned} 8.00002x_1 + 4x_2 + 4x_3 &= 20.00002 \\ 6.00001x_1 + 6x_2 + 3x_3 &= 21.00001 \\ 2.00002x_1 + 8x_2 + x_3 &= 19.00001 \end{aligned} \tag{2.14}$$

which can be viewed as a *perturbed* version of Eqs. (2.13). The reason that a small change in the RHS vector from $\mathbf{b}$ to $\acute{\mathbf{b}}$ (note that $\mathbf{b}$ and $\acute{\mathbf{b}}$ agree through six significant figures) caused such a drastic change in the solution is because the linear systems (2.13) and (2.14) are ill-conditioned (or equivalently, the coefficient matrix A is nearly singular). If A is in fact singular, then, depending on $\mathbf{b}$, the linear system $A\mathbf{x} = \mathbf{b}$ may have no solution or an infinite number of solutions. Thus, if A is nearly singular, we can expect the solution to vary drastically with small changes in $\mathbf{b}$.

Example 2: The linear systems in the preceding example may appear contrived; in fact, they are. Consider this time the linear system

$$\begin{bmatrix} 3 & 8 & 4 & 5 \\ 4 & 9 & 6 & 7 \\ 2 & 5 & 7 & 6 \\ 7 & 7 & 6 & 9 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 20 \\ 26 \\ 20 \\ 29 \end{bmatrix}$$

which is very much like any ordinary system of four equations in four unknowns. The exact solution is $\bar{\mathbf{x}}_{\mathrm{t}} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$, as you may easily verify. The tables below show the solution found by GEM (implemented in FORTRAN 90), indicating the effect on the computed solution of a small change in one of the constants in $\mathbf{b}$ (b_2) or one of the constants in A (a_{32}). The program was run in both single-precision and double-precision mode.

Table 2.1. Effect on $\bar{\mathbf{x}}_{\mathrm{c}}$ of a small change in b_2 (all computations are in single precision)

b_2	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_4$
26.000	1.000019	1.000001	1.000017	0.999973
25.999	1.108888	1.007992	1.095902	0.845158
25.990	2.089976	1.079998	1.959979	-0.549966
25.900	11.899850	1.799989	10.599870	-14.499790
26.001	0.891056	0.992004	0.904050	1.154920
26.010	-0.090031	0.919998	0.039973	2.550044
26.100	-9.899906	0.200007	-8.599918	16.499870

Table 2.2. Effect on $\bar{\mathbf{x}}_{\mathrm{c}}$ of a small change in b_2 (all computations are in double precision)

b_2	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_4$
26.000	1.000000	1.000000	1.000000	1.000000
25.999	1.109000	1.008000	1.096000	0.845000
25.990	2.090000	1.080000	1.960000	-0.550000
25.900	11.900000	1.800000	10.600000	-14.500000
26.001	0.891000	0.992000	0.904000	1.155000
26.010	-0.090000	0.920000	0.040000	2.550000
26.100	-9.900000	0.200000	-8.600000	16.500000

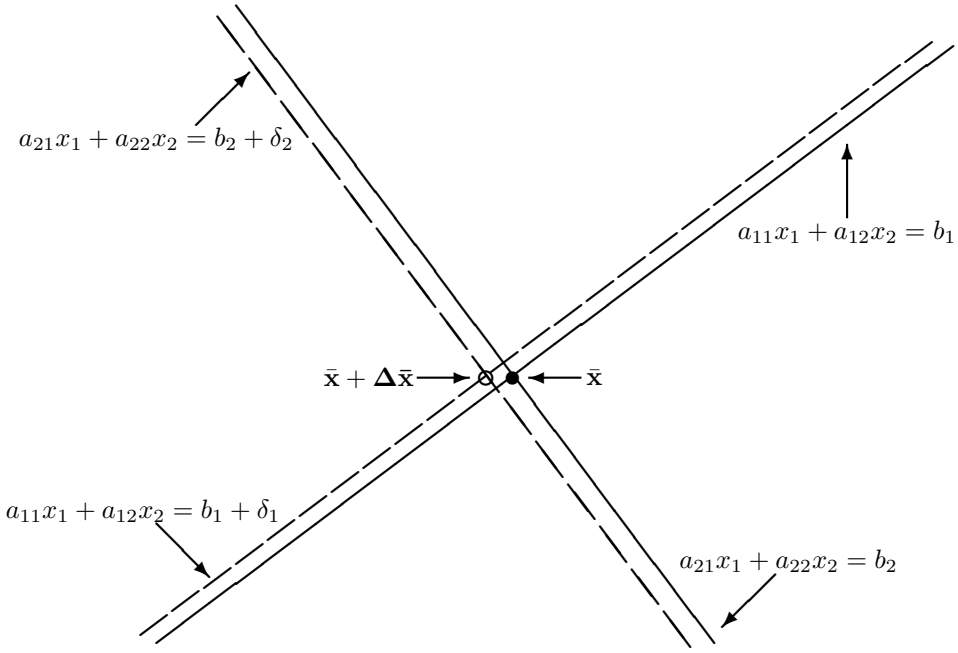
Table 2.3. Effect on $\bar{\mathbf{x}}_{\mathrm{c}}$ of a small change in a_{32} (all computations are in single precision)

a_{32}	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_4$
5.000	1.000019	1.000001	1.000017	0.999973
4.999	1.028063	1.002005	1.025056	0.959910
4.990	1.285727	1.020409	1.255113	0.591819
4.900	4.499938	1.249995	4.124945	-3.999911
5.001	0.972057	0.998004	0.975051	1.039919
5.010	0.725488	0.980392	0.754900	1.392160
5.100	-1.333304	0.833336	-1.083308	4.333292

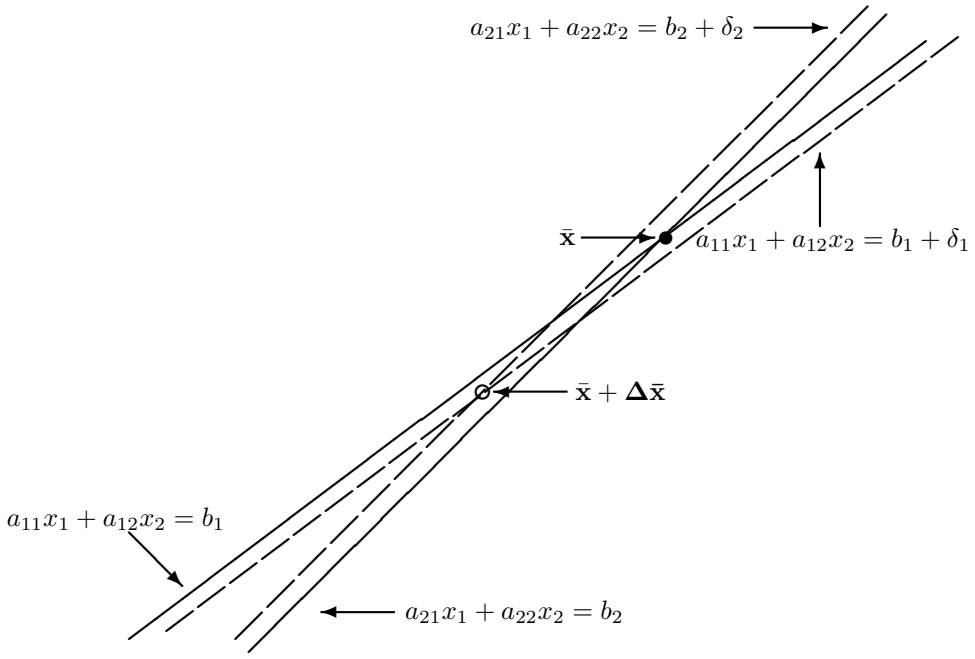
Table 2.4. Effect on $\bar{\mathbf{x}}_{\mathrm{c}}$ of a small change in a_{32} (all computations are in double precision)

a_{32}	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_4$
5.000	1.000000	1.000000	1.000000	1.000000
4.999	1.028056	1.002004	1.025050	0.959920
4.990	1.285714	1.020408	1.255102	0.591837
4.900	4.500000	1.250000	4.125000	-4.000000
5.001	0.972056	0.998004	0.975050	1.039920
5.010	0.725490	0.980392	0.754902	1.392157
5.100	-1.333333	0.833333	-1.083333	4.333333

The change in the solution is clearly out of proportion to the change in $\mathbf{b}$ or A. For an ill-conditioned linear system, relatively small changes in the coefficients a_{ij} or constants b_i produces relatively large changes in $\bar{\mathbf{x}}_c$. Conversely, for a well-conditioned system, small changes in the coefficients or constants produce correspondingly small changes in $\bar{\mathbf{x}}_c$. The figures below depict geometrically this state of affairs for $n = 2$.



Effect of the perturbations on the RHS of a well-conditioned linear system ($n = 2$)



Effect of the perturbations on the RHS of an ill-conditioned linear system ($n = 2$)

2.11 Condition Number of a Matrix

It is clear from the above considerations that in solving the linear system $A\mathbf{x} = \mathbf{b}$, we must have a measure of ill-conditioning or nearness to singularity. This measure is called the *condition number* of a matrix.

In order to arrive at an expression for the condition number of a matrix, we need to define a measure for the 'size' of a vector and a matrix. For some scalar α, the measure of size that we normally use is the *absolute value*, denoted by $|\alpha|$. For vectors and matrices, the measure of size is the *norm*, denoted by $\|\|$.

2.11.1 Vector Norms

A vector norm on the set of all n-dimensional column vectors with real components is a real-valued function which satisfies the following properties for all scalars α and all vectors $\mathbf{x}$ and $\mathbf{y}$:

1. $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0$ iff $\mathbf{x} = \mathbf{0}$ (*The size of the zero vector is zero and the size of any nonzero vector is greater than zero.*)
2. $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$ (*The multiplication of a vector by a scalar multiplies the size of the vector by the magnitude of the scalar.*)
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (*The size of the sum of two vectors is no larger than the sum of the sizes of the two vectors. This property is called the triangle inequality.*)

These are logical properties of size.

The most commonly used vector norms are the so-called ℓ_p norms defined as

$$\|\mathbf{x}\| = \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}} \qquad p \geq 1.$$

Specifically, for $p = 1, 2$ and ∞, we have:

1. the ℓ_1 or one-norm
$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$$
2. the ℓ_2, Euclidean or two-norm (this is the familiar Euclidean length)
$$\|\mathbf{x}\|_2 = \sqrt{{x_1}^2 + {x_2}^2 + \cdots + {x_n}^2}$$
3. the ℓ_∞, $\max$ or infinity norm
$$\|\mathbf{x}\|_\infty = \max\left(|x_1|, |x_2|, \ldots, |x_n|\right)$$

Example: Find $\|\mathbf{s}\|_1$, $\|\mathbf{s}\|_2$ and $\|\mathbf{s}\|_\infty$, given that $\mathbf{s} = \begin{bmatrix} 5 \\ -10 \\ 16 \\ 1 \\ -27 \end{bmatrix}$.

$$\begin{aligned} \|\mathbf{s}\|_1 &= |5| + |-10| + |16| + |1| + |-27| = 59 \\ \|\mathbf{s}\|_2 &= \sqrt{(5)^2 + (-10)^2 + (16)^2 + (1)^2 + (-27)^2} = 33\frac{1}{3} \\ \|\mathbf{s}\|_\infty &= |-27| = 27 \end{aligned}$$

2.11.2 Matrix Norms

A matrix norm on the set of all real $n \times n$ matrices is a real-valued function satisfying the following properties for all scalars α and $n \times n$ matrices A and B:

1. $\|A\| \geq 0$ and $\|A\| = 0$ iff $a_{ij} = 0$, $1 \leq i, j \leq n$
2. $\|\alpha A\| = |\alpha| \|A\|$
3. $\|A + B\| \leq \|A\| + \|B\|$ (*triangle inequality*)
4. $\|AB\| \leq \|A\| \|B\|$ (*consistency*)

Properties 1 through 3 are similar to those for a vector norm; property 4 is called *consistency*.

There are various ways in which a matrix norm may be explicitly defined. Since, in the present case, our primary concern is the linear system $A\mathbf{x} = \mathbf{b}$, it would seem logical to define a matrix norm in terms of the quantity $A\mathbf{x}$.

When a vector $\mathbf{x}$ is multiplied by a matrix A, the result is the vector $A\mathbf{x}$. We can view this effect of A on $\mathbf{x}$ as a transformation from $\mathbf{x}$ to $A\mathbf{x}$. A measure of the 'size' of A is the amount by which $\mathbf{x}$ changes in size relative to its original size when transformed by A, i.e., $\|A\mathbf{x}\|/\|\mathbf{x}\|$, where $\|\|$ is any vector norm. Consequently, we define the norm of A as the *maximum relative change effected by A on all nonzero vectors* $\mathbf{x}$:

$$\|A\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} \tag{2.15}$$

In terms of the 1-, 2- and ∞-norms, we have:

$$\begin{aligned}
\|A\|_1 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_1}{\|\mathbf{x}\|_1} = \max_{\mathbf{x} \neq \mathbf{0}} \sum_{i=1}^{n} |a_{ij}| &= \textit{maximum absolute column sum} \\
\|A\|_2 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_2}{\|\mathbf{x}\|_2} = \sigma_1(A) &= \textit{largest singular value of } A \\
\|A\|_\infty = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \max_{\mathbf{x} \neq \mathbf{0}} \sum_{j=1}^{n} |a_{ij}| &= \textit{maximum absolute row sum}
\end{aligned}$$

The $\|A\|_2$ norm is not readily computed; in its place, another measure, called the *Frobenius norm*, may be used instead.

$$\|A\|_{\mathrm{F}} = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{n} {a_{ij}}^2}$$

Example: Find $\|A\|_1$, $\|A\|_{\mathrm{F}}$ and $\|A\|_\infty$, given that

$$A = \begin{bmatrix} 3 & -4 & 8 \\ -4 & 7 & 5 \\ 4 & 6 & -9 \end{bmatrix}$$

$\|A\|_1 = 22$, $\|A\|_{\mathrm{F}} = 17.32$, $\|A\|_\infty = 19$ (Verify.)

2.11.3 Condition Number of a Matrix

Tables 2.1 through 2.4 given above show that a small perturbation in $\mathbf{b}$ or in A can produce a relatively large change in the computed solution. How a linear system behaves when subjected to perturbations is defined by a quantity called the *condition number* of a matrix.

Consider the linear system $A\mathbf{x} = \mathbf{b}$ and let $\bar{\mathbf{x}}$ be the exact solution. Let $\mathbf{b} + \Delta\mathbf{b}$ be the perturbed RHS and let $\bar{\mathbf{x}} + \Delta\bar{\mathbf{x}}$ be the exact solution to the perturbed system. Then

$$A(\bar{\mathbf{x}} + \Delta\bar{\mathbf{x}}) = \mathbf{b} + \Delta\mathbf{b}.$$

Since $A\bar{\mathbf{x}} = \mathbf{b}$, it follows that $A\Delta\bar{\mathbf{x}} = \Delta\mathbf{b}$ or $\Delta\bar{\mathbf{x}} = A^{-1}\Delta\mathbf{b}$. Applying the definition of a matrix norm [Eq. (2.15)], we have

$$\|A\bar{\mathbf{x}}\| \leq \|A\|\|\bar{\mathbf{x}}\|,$$

or, since $A\bar{\mathbf{x}} = \mathbf{b}$, then

$$\|\mathbf{b}\| \leq \|A\|\|\bar{\mathbf{x}}\| \tag{2.16}$$

Likewise,

$$\|A^{-1}\Delta\mathbf{b}\| \leq \|A^{-1}\|\|\Delta\mathbf{b}\|,$$

or, since $\Delta\bar{\mathbf{x}} = A^{-1}\Delta\mathbf{b}$, then

$$\|\Delta\bar{\mathbf{x}}\| \leq \|A^{-1}\|\|\Delta\mathbf{b}\| \tag{2.17}$$

Multiplying the inequalities (2.16) and (2.17) yields

$$\|\mathbf{b}\|\|\Delta\bar{\mathbf{x}}\| \leq \|A\|\|A^{-1}\|\|\bar{\mathbf{x}}\|\|\Delta\mathbf{b}\| \tag{2.18}$$

Finally, dividing both sides of (2.18) by $\|\bar{\mathbf{x}}\|\|\mathbf{b}\|$, we obtain the very important relation

$$\frac{\|\Delta\bar{\mathbf{x}}\|}{\|\bar{\mathbf{x}}\|} \leq \|A\|\|A^{-1}\|\frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|} \tag{2.19}$$

The quantity $\|\Delta\mathbf{b}\|/\|\mathbf{b}\|$ in (2.19) is the relative change in the RHS of the original linear system, and the quantity $\|\Delta\bar{\mathbf{x}}\|/\|\bar{\mathbf{x}}\|$ is the relative change in the exact solution $\bar{\mathbf{x}}$ brought about by this change in the original RHS. The factor $\|A\|\|A^{-1}\|$ is called the *condition number* of A.

$$\text{cond}(A) = \|A\|\|A^{-1}\|$$

It is clear from (2.19) that if the condition number of A is large, a small change in $\mathbf{b}$ will result in a disproportionately large change in the computed solution. It can be shown that the same effect is produced by a small change in A.

The following facts can be readily established for the condition number of a matrix:

1. $\text{cond}(A) \geq 1$
2. $\text{cond}(I) = 1$
3. A matrix whose condition number is unity is said to be perfectly conditioned.
4. A singular matrix has a condition number of infinity.

The condition number allows us to make precise what we mean when we say that a matrix is ill-conditioned. Recall that the quantity $\|A\mathbf{x}\|/\|\mathbf{x}\|$ represents the amount by which a vector $\mathbf{x}$ changes in size relative to its original size when transformed by A, and that we defined the matrix norm $\|A\|$ to be the *maximum* such change over all nonzero vectors $\mathbf{x}$ [see Eq. (2.15)]. Now the *minimum* change over all nonzero vectors $\mathbf{x}$ is

$$\min_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} = \min_{\mathbf{b}\neq\mathbf{0}} \frac{\|\mathbf{b}\|}{\|A^{-1}\mathbf{b}\|} = \frac{1}{\max\limits_{\mathbf{b}\neq\mathbf{0}} \dfrac{\|A^{-1}\mathbf{b}\|}{\|\mathbf{b}\|}} = \frac{1}{\|A^{-1}\|} \tag{2.20}$$

Dividing Eq. (2.15) by Eq. (2.20) yields

$$\frac{\max\limits_{\mathbf{x}\neq\mathbf{0}} \dfrac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}}{\min\limits_{\mathbf{x}\neq\mathbf{0}} \dfrac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}} = \|A\|\|A^{-1}\| = \text{cond}(A) \tag{2.21}$$

Eq. (2.21) gives us another view of $\mathrm{cond}(A)$, viz., as the ratio of the maximum to the minimum relative change in size effected by A on all nonzero vectors $\mathbf{x}$. A matrix A is ill-conditioned when it transforms vectors of the same size into vectors of radically different sizes. In terms of Eq. (2.21), ill-conditioning is indicated when the difference between the maximum and the minimum values of $\|A\mathbf{x}\|/\|\mathbf{x}\|$ is large. Note that if $\|A\mathbf{x}\|/\|\mathbf{x}\|$ is uniformly large, or uniformly small, for all $\mathbf{x}$, then A is not ill-conditioned.

Example: Consider the matrix

$$A = \begin{bmatrix} 10^{-10} & 0 \\ 0 & 10^{-10} \end{bmatrix}.$$

If this matrix is applied on any vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, the result is $A\mathbf{x} = \begin{bmatrix} 10^{-10}x_1 \\ 10^{-10}x_2 \end{bmatrix}$, and $\|A\mathbf{x}\|/\|\mathbf{x}\| = 10^{-10}$ for all $\mathbf{x}$. Thus, A is not ill-conditioned; in fact, it is perfectly conditioned, since

$$\mathrm{cond}(A) = \|A\|\|A^{-1}\| = \left\| \begin{matrix} 10^{-10} & 0 \\ 0 & 10^{-10} \end{matrix} \right\| \left\| \begin{matrix} 10^{+10} & 0 \\ 0 & 10^{+10} \end{matrix} \right\| = 1$$

where $\|\|$ is any matrix norm. This example also illustrates the fact that the determinant is an unreliable measure of nearness to singularity. The determinant of A is 10^{-20}, which is very small; yet A is perfectly conditioned.

Now, consider the matrix

$$B = \begin{bmatrix} 10^{-10} & 1 \\ 1 & 2 \times 10^{10} \end{bmatrix},$$

and let $\mathbf{u}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{u}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Then $\|B\mathbf{u}_1\|_\infty = 1$ and $\|B\mathbf{u}_2\|_\infty = 2 \times 10^{10}$. We see that B transforms two vectors of unit size to vectors of radically different sizes, indicating that B is ill-conditioned. In fact,

$$\mathrm{cond}(B) = \|B\|_\infty \|B^{-1}\|_\infty = \left\| \begin{matrix} 10^{-10} & 1 \\ 1 & 2 \times 10^{10} \end{matrix} \right\|_\infty \left\| \begin{matrix} 2 \times 10^{10} & -1 \\ -1 & 10^{-10} \end{matrix} \right\|_\infty = (2 \times 10^{10} + 1)^2 \approx 4 \times 10^{20}$$

which is very large, as expected. Note, however, that the determinant of B is equal to 1, which is nowhere near 0, indicating again that it is not a reliable measure of matrix conditioning.

2.12 Jacobi and Gauss-Seidel Methods

Consider the system of linear equations

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 + \cdots + a_{1(n-1)}x_{n-1} + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 + \cdots + a_{2(n-1)}x_{n-1} + a_{2n}x_n &= b_2 \\
a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 + \cdots + a_{3(n-1)}x_{n-1} + a_{3n}x_n &= b_3 \\
a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 + \cdots + a_{4(n-1)}x_{n-1} + a_{4n}x_n &= b_4 \\
&\vdots \quad \vdots \\
a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + a_{n4}x_4 + \cdots + a_{n(n-1)}x_{n-1} + a_{nn}x_n &= b_n
\end{aligned} \tag{2.2}$$

so ordered such that no diagonal element a_{ii} is equal to zero. Eqs. (2.2) may then be rewritten as follows:

$$\begin{aligned}
x_1 &= (b_1 - \quad a_{12}x_2 - a_{13}x_3 - a_{14}x_4 - \cdots - a_{1(n-1)}x_{n-1} - a_{1n}x_n) \,/\, a_{11} \\
x_2 &= (b_2 - a_{21}x_1 - \quad a_{23}x_3 - a_{24}x_4 - \cdots - a_{2(n-1)}x_{n-1} - a_{2n}x_n) \,/\, a_{22} \\
x_3 &= (b_3 - a_{31}x_1 - a_{32}x_2 - \quad a_{34}x_4 - \cdots - a_{3(n-1)}x_{n-1} - a_{3n}x_n) \,/\, a_{33} \\
x_4 &= (b_4 - a_{41}x_1 - a_{42}x_2 - a_{43}x_3 - \quad \cdots - a_{4(n-1)}x_{n-1} - a_{4n}x_n) \,/\, a_{44} \\
&\vdots \\
x_n &= (b_n - a_{n1}x_1 - a_{n2}x_2 - a_{n3}x_3 - a_{n4}x_4 - \cdots - a_{n(n-1)}x_{n-1}) \,/\, a_{nn}
\end{aligned} \tag{2.22}$$

or, more concisely,

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1;j\neq i}^{n} a_{ij} x_j \right) \qquad i = 1, 2, 3, \ldots, n \tag{2.22}$$

Eqs. (2.22) may be interpreted as a rule for transforming a given set of values $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$ on the right into a new set of value on the left. The new set of values on the left will be different from the given set on the right unles the given values are actually the solution to Eqs. (2.2). Thus solving Eqs. (2.2) may be viewed as the problem of finding that set of values $\bar{\mathbf{x}} = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \bar{x}_3 \\ \vdots \\ \bar{x}_n \end{bmatrix}$ which under the transformation (2.22) reproduces itself.

Another way of looking at (2.22) is as the generalization of the MoSS from the case of one equation in one unknown to the case of n linear equations in n unknowns, since if we let

$$F_i(\mathbf{x}) = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1;j\neq i}^{n} a_{ij} x_j \right) \qquad i = 1, 2, 3, \ldots, n \tag{2.23}$$

then Eqs. (2.22) takes on the familiar form of the MoSS:

$$x_i = F_i(x_1, x_2, x_3, \ldots, x_n) \qquad i = 1, 2, 3, \ldots, n$$

Example: Consider the linear system

$$\begin{array}{rcl} 2x_1 - 7x_2 + 4x_3 &= 9 & \\ x_1 + 9x_2 - 6x_3 &= 1 & \Rightarrow \\ -3x_1 + 8x_2 + 5x_3 &= 6 & \end{array} \qquad \begin{array}{rcl} x_1 &= & (9 + 7x_2 - 4x_3) \,/\, 2 \\ x_2 &= & (1 - \ x_1 + 6x_3) \,/\, 9 \\ x_3 &= & (6 + 3x_1 - 8x_2) \,/\, 5 \end{array}$$

Let $\mathbf{x}_{\text{right}} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$; then $\mathbf{x}_{\text{left}}$ will be

$$\begin{array}{rcl} x_1 &= & (9 + 7 * 2 - 4 * 3) \,/\, 2 = \ \ 5.50 \\ x_2 &= & (1 - \ \ 1 \ \ + 6 * 3) \,/\, 9 = \ \ 2.00 \\ x_3 &= & (6 + 3 * 1 - 8 * 2) \,/\, 5 = -1.40 \end{array}$$

which is not $\mathbf{x}_{\text{right}}$. Now let $\mathbf{x}_{\text{right}} = \begin{bmatrix} 4 \\ 1 \\ 2 \end{bmatrix}$; then $\mathbf{x}_{\text{left}}$ will be

$$\begin{array}{rcl} x_1 &= & (9 + 7 * 1 - 4 * 2) \,/\, 2 = 4 \\ x_2 &= & (1 - \ \ 4 \ \ + 6 * 2) \,/\, 9 = 1 \\ x_3 &= & (6 + 3 * 4 - 8 * 1) \,/\, 5 = 2 \end{array}$$

which is $\mathbf{x}_{\text{right}}$. Thus $\bar{\mathbf{x}} = \begin{bmatrix} 4 \\ 1 \\ 2 \end{bmatrix}$.

The transformations indicated in (2.22) may be carried out in two ways. In the first method (*Jacobi method* or *iteration by total steps*), $\mathbf{x}_{\text{left}}$ is solved *completely* in terms of $\mathbf{x}_{\text{right}}$. Using familiar notation, let $\mathbf{x}_{\text{right}}$ be the vector $\mathbf{x}_k = \begin{bmatrix} x_1^k \\ x_2^k \\ x_3^k \\ \vdots \\ x_n^k \end{bmatrix}$ and $\mathbf{x}_{\text{left}}$ be the vector $\mathbf{x}_{k+1} = \begin{bmatrix} x_1^{k+1} \\ x_2^{k+1} \\ x_3^{k+1} \\ \vdots \\ x_n^{k+1} \end{bmatrix}$. Then (2.22) becomes

$$x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1;j\neq i}^{n} a_{ij} x_j^k \right) \qquad i = 1, 2, \ldots, n \tag{2.24}$$

In the other method (*Gauss-Seidel method* or *iteration by single steps*), the *newest* x_i values are used in computing $\mathbf{x}_{k+1}$. Thus (2.22) becomes

$$x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{k+1} - \sum_{j=i+1}^{n} a_{ij} x_j^k \right) \qquad i = 1, 2, \ldots, n \tag{2.25}$$

It is clear that implementing the Jacobi method requires maintaining two vectors, whereas implementing the Gauss-Seidel method requires only one. In both methods, computation is initiated by substituting into (2.24) or (2.25) an initial vector estimate $\mathbf{x}_0$, to obtain a new vector estimate $\mathbf{x}_1$, and so on.

$$\mathbf{x}_0 \xrightarrow{(2.24)\text{or}(2.25)} \mathbf{x}_1 \xrightarrow{(2.24)\text{or}(2.25)} \mathbf{x}_2 \longrightarrow \cdots \longrightarrow \mathbf{x}_k \xrightarrow{(2.24)\text{or}(2.25)} \mathbf{x}_{k+1} \longrightarrow \cdots$$

The sequence of vector estimates may converge to the solution vector $\bar{\mathbf{x}}$, or it may not. In general, the Gauss-Seidel method converges faster than the Jacobi method; however, there are cases where the latter method converges where the former diverges. In any case, we need to address two questions:

1. Under what conditions will the methods converge?
2. When do we stop the iterations?

2.12.1 Convergence of the Jacobi and Gauss-Seidel Methods

We proved in Module 1 that for the case of one equation in one unknown, a sufficient condition for convergence of the MoSS is $|F'(x)| < 1$. For the Jacobi and Gauss-Siedel methods, which are generalized versions of the MoSS for the case of n linear equations in n unknowns, the corresponding sufficient condition for convergence is

$$\sum_{j=1;j\neq i}^{n} \left| \frac{\partial F_i}{\partial x_j} \right| < 1 \qquad i = 1, 2, 3, \ldots, n$$

that is

$$\left| \frac{\partial F_i}{\partial x_1} \right| + \left| \frac{\partial F_i}{\partial x_2} \right| + \cdots + \left| \frac{\partial F_i}{\partial x_{i-1}} \right| + \left| \frac{\partial F_i}{\partial x_{i+1}} \right| + \cdots + \left| \frac{\partial F_i}{\partial x_n} \right| < 1 \qquad i = 1, 2, \ldots, n$$

or, using Eqs. (2.23)

$$\left| \frac{a_{i1}}{a_{ii}} \right| + \left| \frac{a_{i2}}{a_{ii}} \right| + \cdots + \left| \frac{a_{i(i-1)}}{a_{ii}} \right| + \left| \frac{a_{i(i+1)}}{a_{ii}} \right| + \cdots + \left| \frac{a_{in}}{a_{ii}} \right| < 1 \qquad i = 1, 2, \ldots, n$$

which reduces to

$$|a_{ii}| > |a_{i1}| + |a_{i2}| + \cdots + |a_{i(i-1)}| + |a_{i(i+1)}| + \cdots + |a_{in}| \qquad i = 1, 2, \ldots, n$$

or, more concisely

$$|a_{ii}| > \sum_{j=1;j\neq i}^{n} |a_{ij}| \qquad i = 1, 2, 3, \ldots, n \tag{2.26}$$

The inequalities (2.26) are referred to as the *row-sum criterion*. They define the sufficient, but not necessary, condition for convergence of the Jacobi and Gauss-Seidel methods. The strict inequality $>$ can be relaxed somewhat to $\geq$ if the linear system is *irreducible*.

2.12.2 Terminating the Iterations

1. $\left|\dfrac{x_i^{k+1} - x_i^k}{x_i^{k+1}}\right| < \varepsilon_1 \qquad i = 1, 2, 3, \ldots, n$ *or*
2. $\dfrac{1}{n}\displaystyle\sum_{i=1}^{n}\left|\dfrac{x_i^{k+1} - x_i^k}{x_i^{k+1}}\right| < \varepsilon_2$ (absolute deviation test) *or*
3. $\dfrac{1}{n}\sqrt{\displaystyle\sum_{i=1}^{n}\left(\dfrac{x_i^{k+1} - x_i^k}{x_i^{k+1}}\right)^2} < \varepsilon_3$ (root-mean-square test) *and*
4. a limit on the number of iterations, $itmax$

Example 1: Gauss-Seidel method

$$\begin{bmatrix} 1 & -1 & 1 \\ 5 & -4 & 3 \\ 2 & 1 & 1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -4 \\ -12 \\ 11 \end{bmatrix} \qquad \begin{matrix} \text{(a)} \\ \text{(b)} \\ \text{(c)} \end{matrix}$$

(a) TRANSFORM THE GIVEN LINEAR SYSTEM SUCH THAT THE ROW-SUM CRITERION IS SATISFIED

$$\begin{bmatrix} 2 & 1 & 1 \\ -1 & 6 & -1 \\ 1 & -2 & 3 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 11 \\ 34 \\ -12 \end{bmatrix} \qquad \begin{matrix} \Leftarrow \text{(c)} \\ \Leftarrow 2\times\text{(c)} - \text{(b)} \\ \Leftarrow 6\times\text{(a)} - \text{(b)} \end{matrix}$$

(b) REWRITE THE EQUATIONS IN A FORM SUITABLE FOR THE GAUSS-SEIDEL METHOD

$$\begin{aligned} x_1^{k+1} &= \frac{1}{2}(11 - x_2^k - x_3^k) \\ x_2^{k+1} &= \frac{1}{6}(34 + x_1^{k+1} + x_3^k) \\ x_3^{k+1} &= \frac{1}{3}(-12 - x_1^{k+1} + 2x_2^{k+1}) \end{aligned}$$

(c) PERFORM ITERATIONS OF THE GAUSS-SEIDEL METHOD (USE $\varepsilon = 10^{-03}$)

1st iteration: Assume $\mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.

$$\begin{aligned} x_1^1 &= \frac{1}{2}(11 - 0 - 0) = 5.5000 \\ x_2^1 &= \frac{1}{6}(34 + 5.5000 + 0) = 6.5833 \\ x_3^1 &= \frac{1}{3}(-12 - 5.5000 + 2\times 6.5833) = -1.4444 \\ mre &= \frac{1}{3}\left(\left|\frac{5.5000 - 0}{5.5000}\right| + \left|\frac{6.5833 - 0}{6.5833}\right| + \left|\frac{-1.4444 - 0}{-1.4444}\right|\right) = 1.0000 \end{aligned}$$

2nd iteration: $\mathbf{x}_1 = \begin{bmatrix} 5.5000 \\ 6.5833 \\ -1.4444 \end{bmatrix}$.

$$\begin{aligned} x_1^2 &= \frac{1}{2}(11 - 6.5833 + 1.4444) = 2.9306 \\ x_2^2 &= \frac{1}{6}(34 + 2.9306 - 1.4444) = 5.9144 \\ x_3^2 &= \frac{1}{3}(-12 - 2.9306 + 2 \times 5.9144) = -1.0340 \\ mre &= \frac{1}{3}\left(\left|\frac{2.9306 - 5.5000}{2.9306}\right| + \left|\frac{5.9144 - 6.5833}{5.9144}\right| + \left|\frac{-1.0340 + 1.4444}{-1.0340}\right|\right) = 0.4623 \end{aligned}$$

3rd iteration: $\mathbf{x}_2 = \begin{bmatrix} 2.9306 \\ 5.9144 \\ -1.0340 \end{bmatrix}$.

And so on … Below is a summary of the iterations.

k	x_1^k	x_2^k	x_3^k	MRE
0	0.0000	0.0000	0.0000	
1	5.5000	6.5833	-1.4444	1.0000
2	2.9306	5.9144	-1.0340	0.4623
3	3.0598	6.0043	-1.0171	0.0246
4	3.0064	5.9982	-1.0033	0.0108
5	3.0025	5.9999	-1.0009	0.0013
6	3.0005	5.9999	-1.0002	0.0005

Example 2: Gauss-Seidel method

Assume that we apply the Gauss-Seidel method on the linear system in Example 1 *as given*. Then we have:

$$\begin{aligned} x_1^{k+1} &= -4 + x_2^k - x_3^k \\ x_2^{k+1} &= \frac{1}{4}(12 + 5x_1^{k+1} + 3x_3^k) \\ x_3^{k+1} &= 11 - 2x_1^{k+1} - x_2^{k+1} \end{aligned}$$

The tables below show the results for two initial vector estimates.

(a) $\mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

k	x_1^k	x_2^k	x_3^k	MRE
0	0.0000	0.0000	0.0000	
1	-4.0000	-2.0000	21.0000	1.0000
2	-27.0000	-15.0000	80.0000	0.8187
3	-99.0000	-60.7500	269.7500	0.7279

(b) $\mathbf{x}_0 = \begin{bmatrix} 2.9 \\ 6.1 \\ -0.9 \end{bmatrix}$

k	x_1^k	x_2^k	x_3^k	MRE
0	2.9000	6.1000	-0.9000	
1	3.0000	6.0750	-1.0750	0.6675
2	3.1500	6.1312	-1.4312	0.1019
3	3.5625	6.3797	-2.5047	0.1944
4	4.8844	7.2270	-5.9957	0.3234
5	9.2227	10.0315	-17.4769	0.4690

Gauss-Seidel is obviously diverging in both cases.

Example 3:

This type of linear system arises in the numerical solution of elliptic PDEs (to be discussed in Module 3). The coefficient matrix is called *pentadiagonal* and can be quite large (1000×1000 or larger is not uncommon). The RHS in each equation is $-c$, where c is some constant. Note that the linear system satisfies the row-sum criterion. The Jacobi and Gauss-Seidel methods will converge regardless of what initial vector estimate $\mathbf{x}_0$ is used.

$$\left[\begin{array}{ccccc|ccccc|ccccc|ccccc}
-4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
\hline
1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
\hline
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 & 1 \\
\hline
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & -4
\end{array}\right]
\left[\begin{array}{c} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ \hline x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ \hline x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{15} \\ \hline x_{16} \\ x_{17} \\ x_{18} \\ x_{19} \\ x_{20} \end{array}\right]
= -c
\left[\begin{array}{c} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ \hline 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ \hline 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ \hline 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{array}\right]$$

Results for the Jacobi method ($\varepsilon = 10^{-07}$)

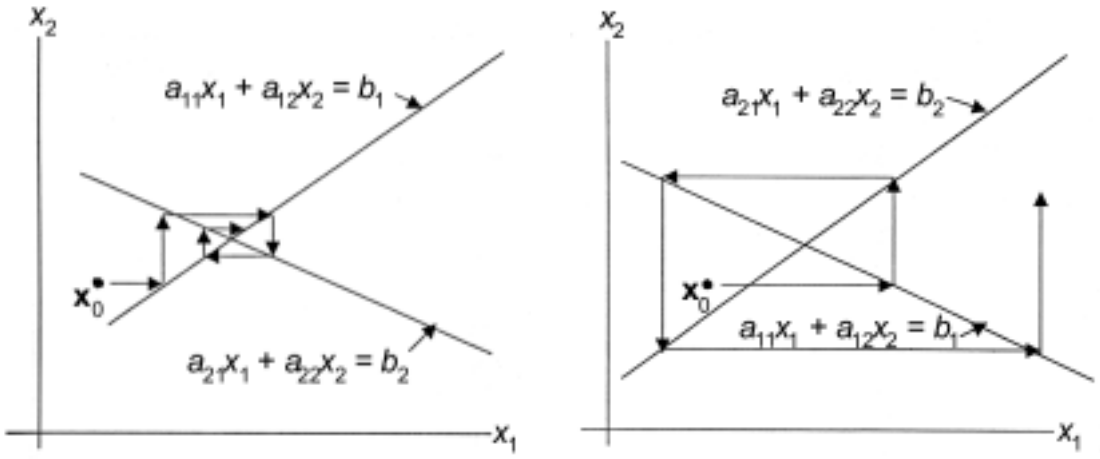
$\mathbf{x}_0 =$	$\mathbf{0}$	$\mathbf{100}$	$\mathbf{-100}$
$\bar{\mathbf{x}} =$	$c\begin{bmatrix} 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \\ \hline 1.246753 \\ 1.857142 \\ 2.038960 \\ 1.857142 \\ 1.246753 \\ \hline 1.246753 \\ 1.857142 \\ 2.038960 \\ 1.857142 \\ 1.246753 \\ \hline 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \end{bmatrix}$	$c\begin{bmatrix} 0.883117 \\ 1.285715 \\ 1.402598 \\ 1.285715 \\ 0.883117 \\ \hline 1.246754 \\ 1.857144 \\ 2.038962 \\ 1.857144 \\ 1.246754 \\ \hline 1.246754 \\ 1.857144 \\ 2.038962 \\ 1.857144 \\ 1.246754 \\ \hline 0.883117 \\ 1.285715 \\ 1.402598 \\ 1.285715 \\ 0.883117 \end{bmatrix}$	$c\begin{bmatrix} 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \\ \hline 1.246753 \\ 1.857142 \\ 2.038960 \\ 1.857142 \\ 1.246753 \\ \hline 1.246753 \\ 1.857142 \\ 2.038960 \\ 1.857142 \\ 1.246753 \\ \hline 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \end{bmatrix}$
$mre =$	$0.885D-07$	$0.959D-07$	$0.989D-07$
$iter =$	82	105	105

Results for the Gauss-Seidel method ($\varepsilon = 10^{-07}$)

$\mathbf{x}_0 =$	$\mathbf{0}$	$\mathbf{100}$	$\mathbf{-100}$
$\bar{\mathbf{x}} =$	$c\begin{bmatrix} 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \\ \hline 1.246753 \\ 1.857142 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 1.246753 \\ 1.857142 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \end{bmatrix}$	$c\begin{bmatrix} 0.883117 \\ 1.285715 \\ 1.402598 \\ 1.285715 \\ 0.883117 \\ \hline 1.246754 \\ 1.857143 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 1.246754 \\ 1.857143 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 0.883117 \\ 1.285714 \\ 1.402598 \\ 1.285714 \\ 0.883117 \end{bmatrix}$	$c\begin{bmatrix} 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \\ \hline 1.246753 \\ 1.857142 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 1.246753 \\ 1.857142 \\ 2.038961 \\ 1.857143 \\ 1.246753 \\ \hline 0.883117 \\ 1.285714 \\ 1.402597 \\ 1.285714 \\ 0.883117 \end{bmatrix}$
$mre =$	$0.796D-07$	$0.743D-07$	$0.765D-07$
$iter =$	44	56	56

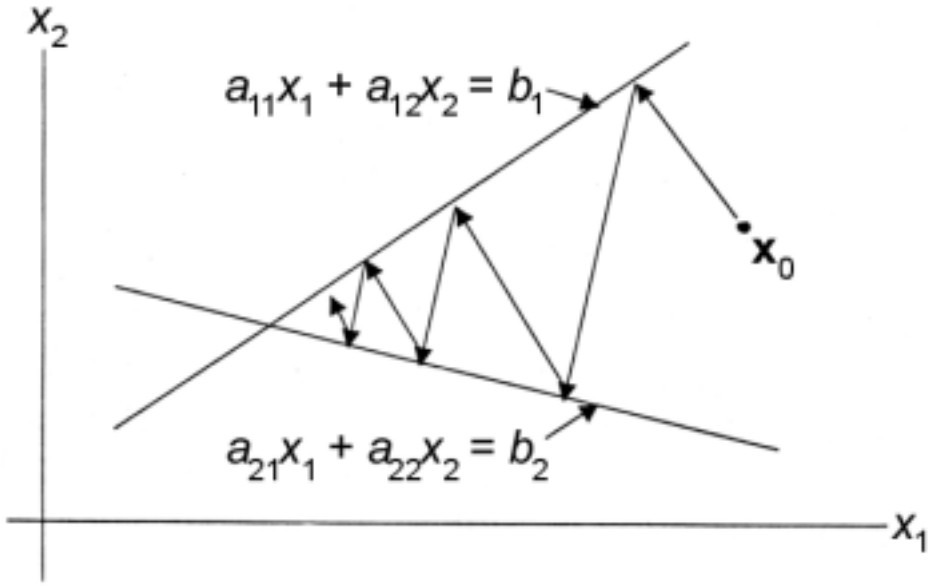
2.13 The Method of Kaczmarz

Consider again the linear system $A\mathbf{x} = \mathbf{b}$ and assume that A is nonsingular, i.e., a unique solution, $\bar{\mathbf{x}}$, exists. While the Jacobi and Gauss-Seidel methods may fail to converge to $\bar{\mathbf{x}}$, the method of Kaczmarz *always* converges to $\bar{\mathbf{x}}$, as indicated in the figure below.



Convergent case Divergent case

The Gauss-Seidel method ($n = 2$)



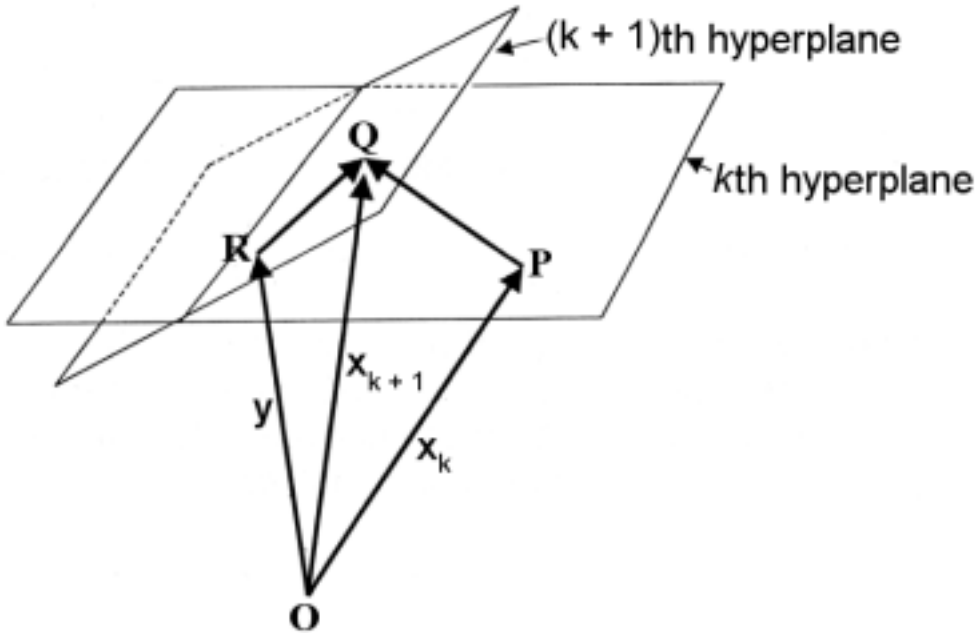
The method of Kaczmarz ($n = 2$)

2.13.1 Derivation of the Iterative Formula for the Method of Kaczmarz

Let $\mathbf{x}$, $\mathbf{y}$, $\mathbf{a}$ be vectors,
$\mathbf{x}_k$, $\mathbf{y}_k$, $\mathbf{a}_k$ be particular *column* vectors,
$\mathbf{x}_{(k)}$, $\mathbf{y}_{(k)}$, $\mathbf{a}_{(k)}$ be particular *row* vectors.

Now, consider the linear system $A\mathbf{x} = \mathbf{b}$, where A is nonsingular with row vectors $\mathbf{a}_{(k)}$, $k = 1, 2, \ldots, n$.

Each equation $\mathbf{a}_{(k)} \cdot \mathbf{x} = b_k$ represents a *hyperplane* in *hyperspace*. Let $\mathbf{x}_k$ be a vector to a point, say P, on the kth hyperplane. Let $\mathbf{x}_{k+1}$ be the vector to the point Q on the $(k+1)$th hyperplane which is reached by moving from P in a direction orthogonal to this hyperplane. Let $\mathbf{y}$ be the vector to *any* other point, say R, on the $(k+1)$th hyperplane.



Then the vectors $\vec{PQ}$ and $\vec{RQ}$ are orthogonal, i.e.,

$$\vec{PQ} \cdot \vec{RQ} = 0$$

or

$$(\mathbf{x}_{k+1} - \mathbf{x}_k) \cdot (\mathbf{x}_{k+1} - \mathbf{y}) = 0 \tag{2.27}$$

Now, since both $\mathbf{x}_{k+1}$ and $\mathbf{y}$ define points on the $(k+1)$th hyperplane, viz., Q and R, the both satisfy the $(k+1)$th equation, i.e.,

$$\mathbf{a}_{(k+1)} \cdot \mathbf{x}_{k+1} = b_{k+1} \tag{2.28}$$

and

$$\mathbf{a}_{(k+1)} \cdot \mathbf{y} = b_{k+1} \tag{2.29}$$

Subtracting (2.29) from (2.28), we obtain

$$\mathbf{a}_{(k+1)} \cdot (\mathbf{x}_{k+1} - \mathbf{y}) = 0 \tag{2.30}$$

Equations (2.27) through (2.30) are true for *any* vector $\mathbf{y}$ to a point on the $(k+1)$th hyperplane. This means that the vectors $\mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{a}_{(k+1)}$ have the same direction and differ only in magnitude. Hence, for some scalar q, we have

$$\mathbf{x}_{k+1} - \mathbf{x}_k = q\mathbf{a}_{(k+1)}$$

or

$$\mathbf{x}_{k+1} = \mathbf{x}_k + q\mathbf{a}_{(k+1)} \tag{2.31}$$

Substituting (2.31) into (2.28) and solving for q, we obtain

$$q = \frac{b_{k-1} - \mathbf{a}_{(k+1)} \cdot \mathbf{x}_k}{\mathbf{a}_{(k+1)} \cdot \mathbf{a}_{(k+1)}} \tag{2.32}$$

Eq. (2.32) can be simplified if we normalize each equation in $A\mathbf{x} = \mathbf{b}$ using the Euclidean norm, since then $\mathbf{a}_{(k+1)} \cdot \mathbf{a}_{(k+1)} = 1$. With this, substituting (2.32) into (2.31) yields the iterative formula for the method of Kaczmarz:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + (b_{k-1} - \mathbf{a}_{(k+1)} \cdot \mathbf{x}_k)\mathbf{a}_{(k+1)} \tag{2.33}$$

In 'scalar' form, Eq. (2.33) becomes

$$\overbrace{x_i^{k+1}}^{\substack{\text{coordinates of the new point} \\ \text{on the } (k+1)\text{th hyperplane}}} = \overbrace{x_i^k}^{\substack{\text{coordinates of the old point} \\ \text{on the } k\text{th hyperplane}}} + \overbrace{\left(b_{k-1} - \sum_{j=1}^{n} a_{(k+1)j}x_j^k\right)}^{\substack{\text{residual when } \mathbf{x}_k \text{ is substituted into } (k+1)\text{th equation}}} \overbrace{a_{(k+1)i}}^{\substack{\text{coefficients of} \\ \text{the } (k+1)\text{th equation}}} \qquad i = 1, 2, \ldots, n \tag{2.33}$$

Each application of Eq. (2.33) is a *subiteration* of the method of Kaczmarz, and it corresponds to a projection from the point $\mathbf{x}_k$ on the kth hyperplane to the point $\mathbf{x}_{k+1}$ on the $(k+1)$th hyperplane in a direction orthogonal to the latter hyperplane. Note that unlike in the Jacobi and Gauss-Seidel methods, where only one element of $\mathbf{x}$ changes per subiteration, this time the entire vector changes. A *full iteration* of the method of Kaczmarz consists in applying Eq. (2.33) n times. Note that except for the first full iteration (which begins from the point $\mathbf{x}_0$ somewhere in hyperspace), every subsequent full iteration begins from a point, say $\mathbf{x}_{\text{old}}$, on the nth hyperplane and ends at a point, say $\mathbf{x}_{\text{new}}$, on the same (nth) hyperplane.

2.13.2 Terminating the Iterations

Let $\mathbf{x}_{\text{old}}$ be the starting point on the nth hyperplane and $\mathbf{x}_{\text{new}}$ be the ending point on the same hyperplane after n applications of Eq. (2.33).

1. $\left|\dfrac{x_i^{\text{new}} - x_i^{\text{old}}}{x_i^{\text{new}}}\right| < \varepsilon_1 \qquad i = 1, 2, 3, \ldots, n$ *or*

2. $\dfrac{1}{n}\displaystyle\sum_{i=1}^{n}\left|\dfrac{x_i^{\text{new}} - x_i^{\text{old}}}{x_i^{\text{new}}}\right| < \varepsilon_2$ (absolute deviation test) *or*

3. $\dfrac{1}{n}\sqrt{\displaystyle\sum_{i=1}^{n}\left(\dfrac{x_i^{\text{new}} - x_i^{\text{old}}}{x_i^{\text{new}}}\right)^2} < \varepsilon_3$ (root-mean-square test) *and*

4. a limit on the number of iterations, $itmax$

Example 1:

$$\begin{bmatrix} 1 & -1 & 1 \\ 5 & -4 & 3 \\ 2 & 1 & 1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -4 \\ -12 \\ 11 \end{bmatrix}$$

NORMALIZE EACH EQUATION USING THE EUCLIDEAN NORM

$$\begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{-1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ \frac{5}{\sqrt{50}} & \frac{-4}{\sqrt{50}} & \frac{3}{\sqrt{50}} \\ \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \frac{-4}{\sqrt{3}} \\ \frac{-12}{\sqrt{50}} \\ \frac{11}{\sqrt{6}} \end{bmatrix}$$

FIRST FULL ITERATION: $\mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

Subiteration 1

$$\begin{aligned}
\alpha &= b_1 - \mathbf{a}_{(1)} \cdot \mathbf{x}_0 = \frac{-4}{\sqrt{3}} - \left[\frac{1}{\sqrt{3}}(0) - \frac{1}{\sqrt{3}}(0) + \frac{1}{\sqrt{3}}(0)\right] = \frac{-4}{\sqrt{3}} \\
x_1^1 &= x_1^0 + \alpha\, a_{11} = 0 + \left(\frac{-4}{\sqrt{3}}\right)\left(\frac{1}{\sqrt{3}}\right) = \frac{-4}{3} \\
x_2^1 &= x_2^0 + \alpha\, a_{12} = 0 + \left(\frac{-4}{\sqrt{3}}\right)\left(\frac{-1}{\sqrt{3}}\right) = \frac{4}{3} \\
x_3^1 &= x_3^0 + \alpha\, a_{13} = 0 + \left(\frac{-4}{\sqrt{3}}\right)\left(\frac{1}{\sqrt{3}}\right) = \frac{-4}{3}
\end{aligned}$$

Subiteration 2

$$\begin{aligned}
\alpha &= b_2 - \mathbf{a}_{(2)} \cdot \mathbf{x}_1 = \frac{-12}{\sqrt{50}} - \left[\frac{5}{\sqrt{50}}\left(\frac{-4}{3}\right) - \frac{4}{\sqrt{50}}\left(\frac{4}{3}\right) + \frac{3}{\sqrt{50}}\left(\frac{-4}{3}\right)\right] = \frac{4}{\sqrt{50}} \\
x_1^2 &= x_1^1 + \alpha\, a_{21} = \frac{-4}{3} + \left(\frac{4}{\sqrt{50}}\right)\left(\frac{5}{\sqrt{50}}\right) = -0.9333 \\
x_2^2 &= x_2^1 + \alpha\, a_{22} = \frac{4}{3} + \left(\frac{4}{\sqrt{50}}\right)\left(\frac{-4}{\sqrt{50}}\right) = 1.0133 \\
x_3^2 &= x_3^1 + \alpha\, a_{23} = \frac{-4}{3} + \left(\frac{4}{\sqrt{50}}\right)\left(\frac{3}{\sqrt{50}}\right) = -1.0933
\end{aligned}$$

Subiteration 3

$$\begin{aligned}
\alpha &= b_3 - \mathbf{a}_{(3)} \cdot \mathbf{x}_2 = \frac{11}{\sqrt{6}} - \left[\frac{2}{\sqrt{6}}(-0.9333) + \frac{1}{\sqrt{6}}(1.0133) + \frac{1}{\sqrt{6}}(-1.0933)\right] = \frac{12.9466}{\sqrt{6}} \\
x_1^3 &= x_1^2 + \alpha\, a_{31} = -0.9333 + \left(\frac{12.9466}{\sqrt{6}}\right)\left(\frac{2}{\sqrt{6}}\right) = 3.3822 \\
x_2^3 &= x_2^2 + \alpha\, a_{32} = 1.0133 + \left(\frac{12.9466}{\sqrt{6}}\right)\left(\frac{1}{\sqrt{6}}\right) = 3.1711 \\
x_3^3 &= x_3^2 + \alpha\, a_{33} = -1.0933 + \left(\frac{12.9466}{\sqrt{6}}\right)\left(\frac{1}{\sqrt{6}}\right) = 1.0645
\end{aligned}$$

$$MRE = \frac{1}{3}\left(\left|\frac{3.3822 - 0}{3.3822}\right| + \left|\frac{3.1711 - 0}{3.1711}\right| + \left|\frac{1.0645 - 0}{1.0645}\right|\right) = 1.0000$$

SECOND FULL ITERATION: $\mathbf{x}_0 = \mathbf{x}_3 = \begin{bmatrix} 3.3822 \\ 3.1711 \\ 1.0645 \end{bmatrix}$

Subiteration 1

$$\begin{aligned}
\alpha &= b_1 - \mathbf{a}_{(1)} \cdot \mathbf{x}_0 = \frac{-4}{\sqrt{3}} - \left[\frac{1}{\sqrt{3}}(3.3822) - \frac{1}{\sqrt{3}}(3.1711) + \frac{1}{\sqrt{3}}(1.0645)\right] = \frac{-5.2756}{\sqrt{3}} \\
x_1^1 &= x_1^0 + \alpha\, a_{11} = 3.3822 + \left(\frac{-5.2756}{\sqrt{3}}\right)\left(\frac{1}{\sqrt{3}}\right) = 1.6237 \\
x_2^1 &= x_2^0 + \alpha\, a_{12} = 3.1711 + \left(\frac{-5.2756}{\sqrt{3}}\right)\left(\frac{-1}{\sqrt{3}}\right) = 4.9296 \\
x_3^1 &= x_3^0 + \alpha\, a_{13} = 1.0645 + \left(\frac{-5.2756}{\sqrt{3}}\right)\left(\frac{1}{\sqrt{3}}\right) = -0.6940
\end{aligned}$$

Subiteration 2

$$\alpha = b_2 - \mathbf{a}_{(2)} \cdot \mathbf{x}_1 = \frac{-12}{\sqrt{50}} - \left[\frac{5}{\sqrt{50}}(1.6237) - \frac{4}{\sqrt{50}}(4.9296) + \frac{3}{\sqrt{50}}(-0.6940)\right] = \frac{1.6819}{\sqrt{50}}$$

$$
\begin{aligned}
x_1^2 &= x_1^1 + \alpha\, a_{21} = 1.6237 + \left(\frac{1.6819}{\sqrt{50}}\right)\left(\frac{5}{\sqrt{50}}\right) = 1.7919 \\
x_2^2 &= x_2^1 + \alpha\, a_{22} = 4.9296 + \left(\frac{1.6819}{\sqrt{50}}\right)\left(\frac{-4}{\sqrt{50}}\right) = 4.7950 \\
x_3^2 &= x_3^1 + \alpha\, a_{23} = -0.6940 + \left(\frac{1.6819}{\sqrt{50}}\right)\left(\frac{3}{\sqrt{50}}\right) = -0.5931
\end{aligned}
$$

Subiteration 3

$$
\begin{aligned}
\alpha &= b_3 - \mathbf{a}_{(3)} \cdot \mathbf{x}_2 = \frac{11}{\sqrt{6}} - \left[\frac{2}{\sqrt{6}}(1.7919) + \frac{1}{\sqrt{6}}(4.7950) + \frac{1}{\sqrt{6}}(-0.5931)\right] = \frac{3.2143}{\sqrt{6}} \\
x_1^3 &= x_1^2 + \alpha\, a_{31} = 1.7919 + \left(\frac{3.2143}{\sqrt{6}}\right)\left(\frac{2}{\sqrt{6}}\right) = 2.8633 \\
x_2^3 &= x_2^2 + \alpha\, a_{32} = 4.7950 + \left(\frac{3.2143}{\sqrt{6}}\right)\left(\frac{1}{\sqrt{6}}\right) = 5.3307 \\
x_3^3 &= x_3^2 + \alpha\, a_{33} = -0.5931 + \left(\frac{3.2143}{\sqrt{6}}\right)\left(\frac{1}{\sqrt{6}}\right) = -0.0574
\end{aligned}
$$

$$
MRE = \frac{1}{3}\left(\left|\frac{2.8633 - 3.3822}{2.8633}\right| + \left|\frac{5.3307 - 3.1711}{5.3307}\right| + \left|\frac{-0.0574 - 1.0645}{-0.0574}\right|\right) = 6.7105
$$

THIRD FULL ITERATION: $\mathbf{x}_0 = \mathbf{x}_3 = \begin{bmatrix} 2.8633 \\ 5.3307 \\ -0.0574 \end{bmatrix}$

And so on… (the solution vector is actually $\begin{bmatrix} 3 \\ 6 \\ -1 \end{bmatrix}$).

Run	ε	$itmax$	$\mathbf{x}_0$	$\bar{\mathbf{x}}_c$	MRE	iterations
1	$1.0D-07$	500	$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 2.999997 \\ 6.000001 \\ -0.999994 \end{bmatrix}$	$0.966409D-07$	305
2	$1.0D-08$	500	$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 3.000000 \\ 6.000000 \\ -0.999999 \end{bmatrix}$	$0.969172D-08$	366
3	$1.0D-09$	500	$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 3.000000 \\ 6.000000 \\ -1.000000 \end{bmatrix}$	$0.971948D-09$	427

Example 2:

$$
\begin{bmatrix} 6 & 2 & 1 & -1 \\ 2 & 4 & 1 & 0 \\ 1 & 1 & 4 & -1 \\ -1 & 0 & -1 & 3 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ -4 \\ 3 \end{bmatrix} \qquad \bar{\mathbf{x}}_t = \begin{bmatrix} 1 \\ 0 \\ -1 \\ 1 \end{bmatrix}
$$

Run	ε	$itmax$	$\mathbf{x}_0$	$\bar{\mathbf{x}}_c$	MRE	iterations
1	$1.0D-06$	100	$\begin{bmatrix}0\\0\\0\\0\end{bmatrix}$	$\begin{bmatrix}0.999999\\0.000001\\-1.000000\\0.999999\end{bmatrix}$	$0.406246D-06$	31
2	$1.0D-07$	100	$\begin{bmatrix}0\\0\\0\\0\end{bmatrix}$	$\begin{bmatrix}1.000000\\0.000000\\-1.000000\\1.000000\end{bmatrix}$	$0.341884D-07$	37

Example 3:

$$\begin{bmatrix}3&8&4&5\\4&9&6&7\\2&5&7&6\\7&7&6&9\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\\x_4\end{bmatrix}=\begin{bmatrix}20\\26\\20\\29\end{bmatrix}\qquad \bar{\mathbf{x}}_t=\begin{bmatrix}1\\1\\1\\1\end{bmatrix}$$

The table below summarizes the results for 7 runs of the method of Kaczmarz for the given linear system, which we found earlier to be ill-conditioned. The initial estimate is the zero vector in each of the runs. The specified tolerance is $1.0D-10$, and it could not be satisfied even after one million full iterations of the method. Note that the computed solutions are nowhere near the true solution, the small mean relative error notwithstanding. A direct method, e.g., GEM, is clearly indicated in this instance.

Iterations	Computed solution vector				MRE
	x_1	x_2	x_3	x_4	
100	0.860048	0.989512	0.876810	1.199136	0.13476D-05
500	0.860054	0.989496	0.876832	1.199129	0.19315D-07
1000	0.860065	0.989497	0.876842	1.199113	0.19314D-07
10000	0.860268	0.989512	0.877020	1.198825	0.19285D-07
100000	0.862278	0.989663	0.878789	1.195964	0.18997D-07
500000	0.870868	0.990307	0.886350	1.183741	0.17772D-07
1000000	0.880856	0.991057	0.895140	1.169529	0.16358D-07

2.13.3 EASY Implementation

```
procedure KACZMARZ(A, n, x, ε, itmax, ks)
//Solves the linear system Ax = b. Upon entry, A contains the augmented matrix [A|b] and x contains
the vector estimate x_0. Upon return, x contains an estimate of the solution vector x̄. If ks = 1, this
estimate satisfies the specified tolerance; else, the tolerance was not satisfied in itmax iterations and x is
an unreliable estimate of x̄.//
  array A(1:n,1:n), x(1:n), d(1:n)
//...Normalize each equation using the Euclidean norm//
  for i ← 1 to n do
    norm ← 0
    for j ← 1 to n do
      norm ← norm + A(i, j) * A(i, j)
    end for
    norm ← √norm
    for j ← 1 to n do
      A(i, j) ← A(i, j)/norm
    end for
  end for                                                    //Normalize ... //
```

```
//...Perform iterations of the method of Kaczmarz//
  for iter ← 1 to itmax do
    for i ← 1 to n do
      d(i) ← 0                                          //initialize difference vector//
    end for
    for k ← 1 to n do                                   //project onto kth hyperplane//
      sum ← 0
      for j ← 1 to n do
        sum ← sum + A(k, j) * x(j)
      end for
      residual ← A(k, n + 1) − sum
      for i ← 1 to n do                                 //find next estimates//
        diff ← residual * A(k, i)
        x(i) ← x(i) + diff
        d(i) ← d(i) + diff
      end for
    end for                                             //project onto ... //
    relerr ← 0
    for i ← 1 to n do
      if x(i) = 0 then relerr ← relerr + |d(i)|
                  else relerr ← relerr + |d(i)/x(i)|
    end for
    if relerr/n < ε then [ks ← 1; return]
  end for                                               //Perform iterations ... //
  ks ← 0
end procedure
```

Some notes on procedure KACZMARZ:

1. Although the method of Kaczmarz always converges, it is still necessary to specify a limit on the iterations because the input A may be singular (in which case there is nothing to converge to if the system is inconsistent), or the convergence may be too slow (in which case another method, say GEM, may well be used instead).

2. Only one vector, $\mathbf{x}$, is used. At the start of a full iteration $\mathbf{x} \equiv \mathbf{x}_{\text{old}}$; at the end of a full iteration $\mathbf{x} \equiv \mathbf{x}_{\text{new}}$. The quantity $\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{old}}$, which is used in the test for termination, is accumulated in the difference vector $\mathbf{d}$.

3. A sample calling sequence for procedure KACZMARZ is:

```
input n, A, x
call KACZMARZ(A, n, x, 10^−10, 1000, ks)
if ks = 0 then [...]                                    //x is unreliable//
          else [...]                                    //x is OK//
```

2.14 Systems of Nonlinear Equations: The Generalized MoSS

Consider the system of n real equations in n real variables $x_1, x_2, x_3, \ldots, x_n$

$$\begin{aligned} f_1(x_1, x_2, x_3, \ldots, x_n) &= 0 \\ f_2(x_1, x_2, x_3, \ldots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, x_3, \ldots, x_n) &= 0 \end{aligned} \tag{2.1}$$

or, more concisely

$$f_i(\mathbf{x}) = 0 \qquad i = 1, 2, 3, \ldots, n \tag{2.1}$$

where $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$. Now, recast Eqs. (2.1) in a form suitable for the MoSS, thus

$$\begin{aligned} x_1 &= F_1(x_1, x_2, x_3, \ldots, x_n) \\ x_2 &= F_2(x_1, x_2, x_3, \ldots, x_n) \\ &\vdots \\ x_n &= F_n(x_1, x_2, x_3, \ldots, x_n) \end{aligned} \tag{2.34}$$

or, more compactly

$$x_i = F_i(\mathbf{x}) \qquad i = 1, 2, 3, \ldots, n \tag{2.34}$$

As with the Jacobi and Gauss-Seidel methods, we can view Eqs. (2.34) as constituting a set of transofrmation equations under which the set of x_i-values on the right transform into a new set on the left. These two sets of values will be different unless the set on the right happens to be *a solution* to Eqs. (2.1). In other words, a solution to (2.1) is a vector whcih reproduces itself under transformations (2.34).

Analogous to the Jacobi and Gauss-Seidel methods, Eqs. (2.34) can be converted into two different iterative schemes, thus:

Jacobi-type iterations

$$x_i^{k+1} = F_i(x_1^k, x_2^k, x_3^k, \ldots, x_n^k) \qquad i = 1, 2, 3, \ldots, n \tag{2.35}$$

Gauss-Seidel-type iterations

$$x_i^{k+1} = F_i(x_1^{k+1}, x_2^{k+1}, x_3^{k+1}, \ldots, x_{i-1}^{k+1}, x_i^k, x_{i+1}^k, \ldots x_n^k) \qquad i = 1, 2, 3, \ldots, n \tag{2.36}$$

In (2.35), the new values on the left are solved *completely* in terms of the old values on the right. In (2.36), the most recent x_i-values are used to compute succeeding ones. It is clear that implementing (2.35) required two distinct vectors, whereas implementing (2.36) requires only one. We refer to (2.35) and (2.36) as the *generalized MoSS*, jacobi-type and Gauss-Seidel-type iterations, respectively.

2.14.1 Conditions for Convergence of the Generalized MoSS

A sufficient condition for convergence of the MoSS when applied to a system of equations according to the iterative schemes (2.35) or (2.36) is

$$\sum_{j=1}^{n} \left| \frac{\partial F_i}{\partial x_j} \right| < 1 \qquad i = 1, 2, 3, \ldots, n \tag{2.37}$$

We have seen earlier that the inequalities (2.37) reduce to the row-sum criterion when the system of equations (2.1) is linear in the unknowns. In this case, convergence depends solely on the nature of the coefficient matrix A (it must be diagonally dominant), and not on the estimates $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_k$ (because the differentiation eliminates teh variables $x_1, x_2, \ldots, x_n$ from the LHS of (2.37)).

When the system of equations (2.1) is nonlinear in the unknowns, the partial derivatives in (2.37) are evaluated at the estimates $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_k$, and the inequalities must be satisfied at these estimates. Thus convergence of the iterative schemes (2.35) and (2.36) depends on the form of the functions $F_i(\mathbf{x})$ that we choose, given Eqs. (2.1), to yield Eqs. (2.34), and on the initial vector estimate $\mathbf{x}_0$ that we use. Recall from Module 1 that this is precisely the case with MoSS when applied to the single equation $f(x) = 0$.

2.14.2 Terminating the Iterations

1. $\left|\dfrac{x_i^{k+1}-x_i^k}{x_i^{k+1}}\right| < \varepsilon_1 \qquad i=1,2,3,\ldots,n$ *or*
2. $\dfrac{1}{n}\displaystyle\sum_{i=1}^{n}\left|\dfrac{x_i^{k+1}-x_i^k}{x_i^{k+1}}\right| < \varepsilon_2$ (absolute deviation test) *or*
3. $\dfrac{1}{n}\sqrt{\displaystyle\sum_{i=1}^{n}\left(\dfrac{x_i^{k+1}-x_i^k}{x_i^{k+1}}\right)^2} < \varepsilon_3$ (root-mean-square test) *and*
4. a limit on the number of iterations, $itmax$

Example 1:

$$\begin{aligned} f_1(x_1,x_2) &= \frac{1}{2}\sin(x_1x_2) - \frac{x_1}{2} - \frac{x_2}{4\pi} = 0 \\ f_2(x_1,x_2) &= \left(1-\frac{1}{4\pi}\right)(e^{2x_1}-e) - 2e\,x_1 + \frac{e\,x_2}{\pi} = 0 \end{aligned}$$

(a) CAST THE EQUATIONS IN A FORM SUITABLE FOR THE GENERALIZED MOSS

$$\begin{aligned} x_1 &= \underbrace{\sin(x_1x_2) - \frac{x_2}{4\pi}}_{F_1(x_1,x_2)} \\ x_2 &= \underbrace{-\left(\pi-\frac{1}{4}\right)(e^{2x_1-1}-1) + 2\pi\,x_1}_{F_2(x_1,x_2)} \end{aligned}$$

(b) PERFORM G-S TYPE ITERATIONS OF THE GENERALIZED MOSS (USE $\varepsilon = 10^{-04}$)

1st iteration: Assume $\mathbf{x}_0 = \begin{bmatrix} 0.4 \\ 3.0 \end{bmatrix}$.

$$\begin{aligned} x_1^1 &= F_1(x_1^0,x_2^0) = \sin(0.4\times 3.0) - \frac{3.0}{2\pi} = 0.4546 \\ x_2^1 &= F_2(x_1^1,x_2^0) = -\left(\pi-\frac{1}{4}\right)(e^{2(0.4546)-1}-1) + 2\pi(0.4546) = 3.1073 \\ MRE &= \frac{1}{2}\left(\left|\frac{0.4546-0.4}{0.4546}\right| + \left|\frac{3.1073-3.0}{3.1073}\right|\right) = 0.0773 \end{aligned}$$

2nd iteration: $\mathbf{x}_1 = \begin{bmatrix} 0.4546 \\ 3.1073 \end{bmatrix}$.

$$\begin{aligned} x_1^2 &= F_1(x_1^1,x_2^1) = \sin(0.4546\times 3.1073) - \frac{3.1073}{2\pi} = 0.4930 \\ x_2^2 &= F_2(x_1^2,x_2^1) = -\left(\pi-\frac{1}{4}\right)(e^{2(0.4930)-1}-1) + 2\pi(0.4930) = 3.1378 \\ MRE &= \frac{1}{2}\left(\left|\frac{0.4930-0.4546}{0.4930}\right| + \left|\frac{3.1378-3.1073}{3.1378}\right|\right) = 0.0438 \end{aligned}$$

3rd iteration: $\mathbf{x}_1 = \begin{bmatrix} 0.4930 \\ 3.1378 \end{bmatrix}$.

And so on... Below is a summary of the iterations.

k	x_1^k	x_2^k	MRE
0	0.4000	3.0000	
1	0.4546	3.1073	0.0773
2	0.4930	3.1378	0.0438
3	0.5003	3.1418	0.0079
4	0.5000	3.1416	0.0004
5	0.5000	3.1416	0.0000

Example 2:

The curves $(x-2)^2+(y-2)^2-1=0$ and $x^3y-8=0$ intersect at two points, one of which is the point $(2,1)$.

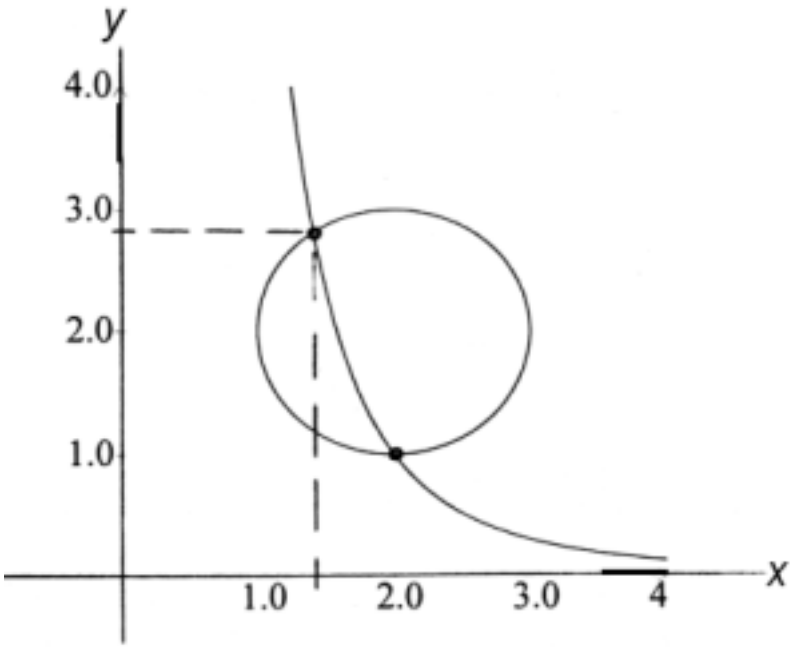
(a) Obtain initial estimates (x_0, y_0) for the other point of intersection.

(b) Cast the equations in a form suitable for the generalized MoSS and show that the conditions for convergence of the method are satisfied at (x_0, y_0).

(c) Use the generalized MoSS, Gauss-Seidel type iterations, to find the desired point with a MRE $< 10^{-04}$.

Solution

(a) PLOT THE GIVEN EQUATIONS TO FIND (x_0, y_0)



From the plot, we take $(x_0, y_0) = (1.4, 2.9)$.

(b) LET

$$x = F_1(x,y) = \left(\frac{8}{y}\right)^{\frac{1}{3}}$$

$$y = F_2(x,y) = \left[1-(x-2)^2\right]^{\frac{1}{2}} + 2$$

Then at $(x_0, y_0) = (1.4, 2.9)$:

$$\begin{aligned}
\left|\frac{\partial F_1}{\partial x}\right| + \left|\frac{\partial F_1}{\partial y}\right| &= 0 + \left|\frac{1}{3}\left(\frac{8}{y}\right)^{\frac{2}{3}}(-8y^{-2})\right| = \left|\frac{-8}{3\left(\frac{8}{2.9}\right)^{\frac{2}{3}}(2.9)^2}\right| = 0.16 < 1 \\
\left|\frac{\partial F_2}{\partial x}\right| + \left|\frac{\partial F_2}{\partial y}\right| &= \left|\frac{1}{2}\left[1-(x-2)^2\right]^{-\frac{1}{2}}\left[-2(x-2)\right]\right| + 0 = \left|\frac{-(1.4-2)}{\sqrt{1-(1.4-2)^2}}\right| = 0.75 < 1
\end{aligned}$$

Therefore, the conditions for convergence of the generalized MoSS are satisfied.

(c) PERFORM G-S TYPE ITERATIONS UNTIL MRE $< 10^{-04}$

1st iteration: $(x_0, y_0) = (1.4, 2.9)$

$$\begin{aligned}
x_1 &= F_1(x_0, y_0) = \left(\frac{8}{2.9}\right)^{\frac{1}{3}} = 1.4025 \\
y_1 &= F_2(x_1, y_0) = \left[1-(1.4025-2)^2\right]^{\frac{1}{2}} + 2 = 2.8019 \\
MRE &= \frac{1}{2}\left(\left|\frac{1.4025-1.4}{1.4025}\right| + \left|\frac{2.8019-2.9}{2.8019}\right|\right) = 0.0184
\end{aligned}$$

2nd iteration: $(x_1, y_1) = (1.4025, 2.8019)$

And so on... Below is a summary of the iterations.

k	x_k	y_k	MRE
0	1.4000	2.9000	
1	1.4025	2.8019	0.0184
2	1.4187	2.8137	0.0078
3	1.4167	2.8122	0.0010
4	1.4169	2.8124	0.0001
5	1.4169	2.8124	0.0000

Exercise: Show that the generalized MoSS will diverge for the form of the equations

$$\begin{aligned}
x &= F_1(x, y) = \sqrt{1-(y-2)^2} + 2 \\
y &= F_2(x, y) = \frac{8}{x^3}
\end{aligned}$$

2.15 Systems of Nonlinear Equations: The Newton-Raphson Method

Consider the system of n real equations in n real variables $x_1, x_2, x_3, \ldots, x_n$

$$\begin{aligned}
f_1(x_1, x_2, x_3, \ldots, x_n) &= 0 \\
f_2(x_1, x_2, x_3, \ldots, x_n) &= 0 \\
&\vdots \\
f_n(x_1, x_2, x_3, \ldots, x_n) &= 0
\end{aligned} \tag{2.1}$$

or, more concisely

$$f_i(\mathbf{x}) = 0 \qquad i = 1, 2, 3, \ldots, n \tag{2.1}$$

where $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$. Now assume that $\mathbf{x}_k = \begin{bmatrix} x_1^k \\ x_2^k \\ x_3^k \\ \vdots \\ x_n^k \end{bmatrix}$ is the kth iteration estimate of the solution vector $\bar{\mathbf{x}} = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \\ \bar{x}_3 \\ \vdots \\ \bar{x}_n \end{bmatrix}$. Expanding $f_i(\mathbf{x})$ about $\mathbf{x}_k$ in a Taylor series, we obtain

$$f_i(\mathbf{x}) = f_i(\mathbf{x}_k) + \sum_{p=1}^{\infty} \left[\frac{1}{p!} \sum_{j=1}^{n} \frac{\partial^{(p)} f_i(\mathbf{x}_k)}{\partial {x_j}^{(p)}} (x_j - x_j^k)^p \right] \qquad i = 1, 2, \ldots, n \tag{2.38}$$

where $\dfrac{\partial^{(p)} f_i(\mathbf{x}_k)}{\partial {x_j}^{(p)}}$ is the pth partial derivative of the function $f_i(\mathbf{x}_k)$ with respect to the variable x_j evaluated at $\mathbf{x}_k$. Taking $p = 1$ only in Eqs. (2.38), we obtain the first order approximations

$$f_i(\mathbf{x}) \approx f_i(\mathbf{x}_k) + \sum_{j=1}^{n} \frac{\partial f_i(\mathbf{x}_k)}{\partial x_j} (x_j - x_j^k) \qquad i = 1, 2, \ldots, n \tag{2.39}$$

Now, evaluating Eqs. (2.39) at $\mathbf{x} = \bar{\mathbf{x}}$, we get

$$f_i(\bar{\mathbf{x}}) = 0 \approx f_i(\mathbf{x}_k) + \sum_{j=1}^{n} \frac{\partial f_i(\mathbf{x}_k)}{\partial x_j} (\bar{x}_j - x_j^k) \qquad i = 1, 2, \ldots, n \tag{2.40}$$

or, in expanded form

$$\begin{aligned}
\frac{\partial f_1(\mathbf{x}_k)}{\partial x_1}(\bar{x}_1 - x_1^k) + \frac{\partial f_1(\mathbf{x}_k)}{\partial x_2}(\bar{x}_2 - x_2^k) + \cdots + \frac{\partial f_1(\mathbf{x}_k)}{\partial x_n}(\bar{x}_n - x_n^k) &\approx -f_1(\mathbf{x}_k) \\
\frac{\partial f_2(\mathbf{x}_k)}{\partial x_1}(\bar{x}_1 - x_1^k) + \frac{\partial f_2(\mathbf{x}_k)}{\partial x_2}(\bar{x}_2 - x_2^k) + \cdots + \frac{\partial f_2(\mathbf{x}_k)}{\partial x_n}(\bar{x}_n - x_n^k) &\approx -f_2(\mathbf{x}_k) \\
&\vdots \\
\frac{\partial f_n(\mathbf{x}_k)}{\partial x_1}(\bar{x}_1 - x_1^k) + \frac{\partial f_n(\mathbf{x}_k)}{\partial x_2}(\bar{x}_2 - x_2^k) + \cdots + \frac{\partial f_n(\mathbf{x}_k)}{\partial x_n}(\bar{x}_n - x_n^k) &\approx -f_n(\mathbf{x}_k)
\end{aligned} \tag{2.40}$$

Relations (2.40) approximate a system of n linear equations in the n unknowns $e_i^k = \bar{x}_i - x_i^k$, $i = 1, 2, \ldots, n$, where e_i^k is the true error of the kth iteration estimate x_i^k. This linear system cannot be solved for the unknowns e_i^k because strict equality does nopt hold between the LHS and RHS in (2.40). So that we can write '$=$' instead of '$\approx$' in (2.40), we replace the true errors $e_i^k = \bar{x}_i - x_i^k$, by the approximations $\delta_i^k = x_i^{k+1} - x_i^k$, $i = 1, 2, \ldots, n$ to yield

$$\begin{aligned}
\frac{\partial f_1(\mathbf{x}_k)}{\partial x_1}\delta_1^k + \frac{\partial f_1(\mathbf{x}_k)}{\partial x_2}\delta_2^k + \cdots + \frac{\partial f_1(\mathbf{x}_k)}{\partial x_n}\delta_n^k &= -f_1(\mathbf{x}_k) \\
\frac{\partial f_2(\mathbf{x}_k)}{\partial x_1}\delta_1^k + \frac{\partial f_2(\mathbf{x}_k)}{\partial x_2}\delta_2^k + \cdots + \frac{\partial f_2(\mathbf{x}_k)}{\partial x_n}\delta_n^k &= -f_2(\mathbf{x}_k) \\
&\vdots \\
\frac{\partial f_n(\mathbf{x}_k)}{\partial x_1}\delta_1^k + \frac{\partial f_n(\mathbf{x}_k)}{\partial x_2}\delta_2^k + \cdots + \frac{\partial f_n(\mathbf{x}_k)}{\partial x_n}\delta_n^k &= -f_n(\mathbf{x}_k)
\end{aligned} \tag{2.41}$$

Eqs. (2.41) constitute a linear system that can be solved for the unknowns $\delta_i^k = x_i^{k+1} - x_i^k$, $i = 1, 2, \ldots, n$ using some appropriate method, sucj as GEM. With the δ_i^k known, the new estimates are readily obtained from Eqs. (2.42):

$$x_i^{k+1} = x_i^k + \delta_i^k \qquad i = 1, 2, 3, \ldots, n \tag{2.42}$$

2.15.1 Algorithm

1. Choose an initial vector estimate $\mathbf{x}_k = \left[x_1^k x_2^k x_3^k \vdots x_n^k\right]$, $k = 0$, such that $\mathbf{x}_0$ is near $\bar{\mathbf{x}}$.

2. Generate the matrix of coefficients Φ_k and the vector of constants $\mathbf{f}_k$ where

$$\Phi_k = \begin{bmatrix} \varphi_{11}^k & \varphi_{12}^k & \varphi_{13}^k & \cdots & \varphi_{1n}^k \\ \varphi_{21}^k & \varphi_{22}^k & \varphi_{23}^k & \cdots & \varphi_{2n}^k \\ \varphi_{31}^k & \varphi_{32}^k & \varphi_{33}^k & \cdots & \varphi_{3n}^k \\ \vdots & & & & \\ \varphi_{n1}^k & \varphi_{n2}^k & \varphi_{n3}^k & \cdots & \varphi_{nn}^k \end{bmatrix} \qquad \text{and} \qquad \mathbf{f}_k = \begin{bmatrix} f_1^k \\ f_2^k \\ f_3^k \\ \vdots \\ f_n^k \end{bmatrix}$$

   where

   $\varphi_{ij}^k \equiv \dfrac{\partial f_i(\mathbf{x}_k)}{\partial x_j}$ is the partial derivative of the function $f_i(\mathbf{x})$ with respect to x_j evaluated at the kth iteration estimate $\mathbf{x}_k$

   $f_i^k \equiv f_i(\mathbf{x}_k)$ is the function $f_i(\mathbf{x})$ evaluated at $\mathbf{x}_k$

3. Solve the linear system $\Phi_k \Delta\mathbf{x}_k = -\mathbf{f}_k$ for the vector of corrections $\Delta\mathbf{x}_k = \begin{bmatrix} \delta_1^k \\ \delta_2^k \\ \vdots \\ \delta_n^k \end{bmatrix}$.

4. Calculate the new estimate $\mathbf{x}_{k+1} = \begin{bmatrix} x_1^{k+1} \\ x_2^{k+1} \\ \vdots \\ x_n^{k+1} \end{bmatrix}$, $x_i^{k+1} = x_i^k + \delta_i^k$, $i = 1, 2, 3, \ldots, n$.

5. Repeat steps (2), (3) and (4) until one of the following error criteria is satisfied

   (a) $\left|\dfrac{\delta_i^k}{x_i^{k+1}}\right| < \varepsilon_1 \qquad i = 1, 2, 3, \ldots, n$

   (b) $\dfrac{1}{n}\displaystyle\sum_{i=1}^{n}\left|\frac{\delta_i^k}{x_i^{k+1}}\right| < \varepsilon_2$

   (c) $\dfrac{1}{n}\sqrt{\displaystyle\sum_{i=1}^{n}\left(\frac{\delta_i^k}{x_i^{k+1}}\right)^2} < \varepsilon_3$

   or the number of iterations is already $itmax$.

2.15.2 Conditions for Convergence of the Newton-Raphson Method

1. The initial estimate $\mathbf{x}_0$ must be 'near' $\bar{\mathbf{x}}$.

2. The components of Φ, i.e., the partial derivatives $\dfrac{\partial f_i(\mathbf{x})}{\partial x_j}$, $1 \le i, j \le n$, are continuous in a neighborhood of $\bar{\mathbf{x}}$.

3. $\det[\Phi(\bar{\mathbf{x}})] \neq 0$

Example 1:

$$\begin{aligned} f_1(x_1, x_2) &= \frac{1}{2}\sin(x_1 x_2) - \frac{x_1}{2} - \frac{x_2}{4\pi} = 0 \\ f_2(x_1, x_2) &= \left(1 - \frac{1}{4\pi}\right)(e^{2x_1} - e) - 2e\, x_1 + \frac{e\, x_2}{\pi} = 0 \end{aligned}$$

(a) GENERATE COMPONENTS OF THE COEFFICIENT MATRIX Φ

$$\begin{aligned} \varphi_{11} &= \frac{\partial f_1}{\partial x_1} = \frac{x_2}{2}\cos(x_1 x_2) - \frac{1}{2} \\ \varphi_{12} &= \frac{\partial f_1}{\partial x_2} = \frac{x_1}{2}\cos(x_1 x_2) - \frac{1}{4\pi} \\ \varphi_{21} &= \frac{\partial f_2}{\partial x_1} = 2\left(1 - \frac{1}{4\pi}\right)e^{2x_1} - 2e \\ \varphi_{22} &= \frac{\partial f_2}{\partial x_2} = \frac{e}{\pi} \end{aligned}$$

(b) PERFORM ITERATIONS OF THE NEWTON-RAPHSON METHOD (USE $\varepsilon = 10^{-04}$)

1st iteration: Assume $\mathbf{x}_0 = \begin{bmatrix} 0.4 \\ 3.0 \end{bmatrix}$

$$\begin{aligned} \varphi_{11}^0 &= \frac{3.0}{2}\cos(0.4 \times 3.0) - \frac{1}{2} = 0.0435 \\ \varphi_{12}^0 &= \frac{0.4}{2}\cos(0.4 \times 3.0) - \frac{1}{4\pi} = -0.0071 \\ \varphi_{21}^0 &= 2\left(1 - \frac{1}{4\pi}\right)e^{2\times 0.4} - 2e = -1.3397 \\ \varphi_{22}^0 &= \frac{e}{\pi} = \frac{2.71828}{3.14159} = 0.8653 \\ f_1^0 &= \frac{1}{2}\sin(0.4 \times 3.0) - \frac{0.4}{2} - \frac{3.0}{4\pi} = 0.0273 \\ f_2^0 &= \left(1 - \frac{1}{4\pi}\right)(e^{2\times 0.4} - e) - 2e(0.4) + \frac{e(3.0)}{\pi} = -0.0324 \end{aligned}$$

$$\begin{array}{ccc} \Phi_0 & \Delta\mathbf{x}_0 & = & -\mathbf{f}_0 \\ \begin{bmatrix} 0.0435 & -0.0071 \\ -1.3397 & 0.8653 \end{bmatrix} & \begin{bmatrix} \delta_1^0 \\ \delta_2^0 \end{bmatrix} & = & \begin{bmatrix} -0.0273 \\ +0.0324 \end{bmatrix} \end{array}$$

$\delta_1^0 = -0.8305$ $\qquad$ $\delta_2^0 = -1.2485$ $\qquad$ (Verify.)

$$\begin{aligned} x_1^1 &= x_1^0 + \delta_1^0 = 0.4 - 0.8305 = -0.4305 \\ x_2^1 &= x_2^0 + \delta_2^0 = 3.0 - 1.2485 = 1.7515 \\ MRE &= \frac{1}{2}\left(\left|\frac{-0.8305}{-0.4305}\right| + \left|\frac{-1.2485}{1.7515}\right|\right) = 1.3209 \end{aligned}$$

2nd iteration: $\mathbf{x}_1 = \begin{bmatrix} -0.4305 \\ 1.7515 \end{bmatrix}$

$$\begin{aligned} \varphi_{11}^1 &= \frac{1.7515}{2}\cos(-0.4305 \times 1.7515) - \frac{1}{2} = 0.1383 \\ \varphi_{12}^1 &= \frac{-0.4305}{2}\cos(-0.4305 \times 1.7515) - \frac{1}{4\pi} = -0.2365 \\ \varphi_{21}^1 &= 2\left(1 - \frac{1}{4\pi}\right)e^{2(-0.4305)} - 2e = -4.6584 \end{aligned}$$

$$\begin{aligned}
\varphi_{22}^1 &= \frac{e}{\pi} = \frac{2.71828}{3.14159} = 0.8653 \\
f_1^1 &= \frac{1}{2}\sin(-0.4305 \times 1.7515) - \frac{-0.4305}{2} - \frac{1.7515}{4\pi} = -0.2644 \\
f_2^1 &= \left(1 - \frac{1}{4\pi}\right)(e^{2(-0.4305)} - e) - 2e(-0.4305) + \frac{e(1.7515)}{\pi} = 1.7432
\end{aligned}$$

$$\begin{array}{ccc} \Phi_1 & \Delta\mathbf{x}_1 & = & -\mathbf{f}_1 \\ \left[\begin{array}{cc} 0.1383 & -0.2365 \\ -4.6584 & 0.8653 \end{array}\right] & \left[\begin{array}{c} \delta_1^1 \\ \delta_2^1 \end{array}\right] & = & \left[\begin{array}{c} +0.2664 \\ -1.7432 \end{array}\right] \end{array}$$

$$\delta_1^1 = 0.1851 \qquad \delta_2^1 = -1.0183 \qquad \text{(Verify.)}$$

$$\begin{aligned}
x_1^2 &= x_1^1 + \delta_1^1 = -0.4305 + 0.1851 = -0.2455 \\
x_2^2 &= x_2^1 + \delta_2^1 = 1.7515 - 1.0183 = 0.7332 \\
MRE &= \frac{1}{2}\left(\left|\frac{0.1851}{-0.2455}\right| + \left|\frac{-1.0183}{0.7332}\right|\right) = 1.0714
\end{aligned}$$

3rd iteration: $\mathbf{x}_2 = \begin{bmatrix} -0.2455 \\ 0.7332 \end{bmatrix}$

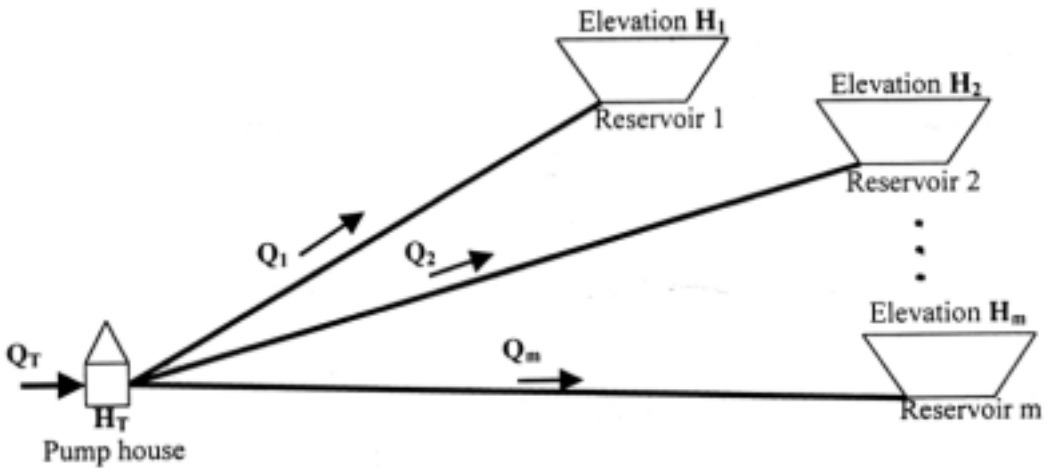
And so on... Below is a summary of the iterations.

k	x_1^k	x_2^k	MRE
0	0.4000	3.0000	
1	-0.4305	1.7515	1.3209
2	-0.2455	0.7332	1.0714
3	-0.2614	0.6189	0.1228
4	-0.2606	0.6225	0.0044
5	-0.2606	0.6225	0.0000

Hence: $\bar{\mathbf{x}} = \begin{bmatrix} -0.2606 \\ 0.6225 \end{bmatrix}$.

The initial vector estimate $\mathbf{x}_0 = \begin{bmatrix} 0.6 \\ 3.0 \end{bmatrix}$ yields the solution vector $\bar{\mathbf{x}} = \begin{bmatrix} \frac{1}{2} \\ \pi \end{bmatrix}$. (Verify). With nonlinear systems, the solution is not necessarily unique.

Example 2: Newton-Raphson to solve parallel reservoir problem



A system of parallel reservoirs

1. Given quantities
   (a) Total available discharge, Q_{T}, cfs
   (b) Elevation at each reservoir, H_i, ft $\quad i = 1, 2, \ldots, m$
   (c) Length of each pipe, L_i, ft $\quad i = 1, 2, \ldots, m$
   (d) Diameter of each pipe, D_i, ft $\quad i = 1, 2, \ldots, m$
   (e) Absolute roughness for each pipe, e_i, ft $\quad i = 1, 2, \ldots, m$
2. Required attributes
   (a) Elevation at each reservoir, Q_i, cfs $\quad i = 1, 2, \ldots, m$
   (b) Total head at the pump house, H_{T}, ft
3. Governing equations
   (a) Energy equations

$$H_i = -f_i \frac{L_i}{D_i} \frac{\left(\dfrac{4Q_i}{\pi\, {D_i}^2}\right)^2}{2g} + H_{\mathrm{T}} \qquad i = 1, 2, \ldots, m$$

   where the friction factor f_i is given by the empirical equation

$$\frac{1}{\sqrt{f_i}} = 2\log_{10}\left(\frac{D_i}{e_i}\right) + 1.14$$

   (b) Continuity equation

$$Q_{\mathrm{T}} = Q_1 + Q_2 + \cdots + Q_m$$

4. Solution
   (a) WRITE THE EQUATIONS IN A FORM SUITABLE FOR THE NEWTON-RAPHSON METHOD
   Let:

$$\begin{aligned} f_i &= \frac{1}{\left(2\log_{10}\left(\dfrac{D_i}{e_i}\right) + 1.14\right)^2} \\ c_i &= f_i \frac{L_i}{D_i} \frac{\left(\dfrac{4}{\pi\, {D_i}^2}\right)^2}{2g} = \frac{8 f_1 L_i}{g\pi^2 {D_i}^5} \end{aligned}$$

   Then:

$$\left.\begin{aligned} f_1(Q_1, Q_2, \ldots, Q_m, H_{\mathrm{T}}) &= c_1 {Q_1}^2 - H_{\mathrm{T}} + H_1 = 0 \\ f_2(Q_1, Q_2, \ldots, Q_m, H_{\mathrm{T}}) &= c_2 {Q_2}^2 - H_{\mathrm{T}} + H_2 = 0 \\ &\vdots \\ f_m(Q_1, Q_2, \ldots, Q_m, H_{\mathrm{T}}) &= c_m {Q_m}^2 - H_{\mathrm{T}} + H_m = 0 \end{aligned}\right\} \text{energy equations}$$

$$f_n(Q_1, Q_2, \ldots, Q_m, H_{\mathrm{T}}) = Q_1 + Q_2 + \cdots + Q_m - Q_{\mathrm{T}} = 0 \;\} \text{ continuity equation}$$

   (b) FORM THE AUGMENTED MATRIX $[\Phi_k | - \mathbf{f}_k]$

$$[\Phi_k | - \mathbf{f}_k] = \left[\begin{array}{cccccc|c} 2c_1 Q_1^k & 0 & 0 & \cdots & 0 & -1 & -f_1^k \\ 0 & 2c_2 Q_2^k & 0 & \cdots & 0 & -1 & -f_2^k \\ 0 & 0 & 2c_3 Q_3^k & \cdots & 0 & -1 & -f_3^k \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 2c_m Q_m^k & -1 & -f_m^k \\ 1 & 1 & 1 & \cdots & 1 & 0 & -f_n^k \end{array}\right]$$

The superscript k means that the coefficients φ_{ij} and function values f_i are evaluated at the kth iteration vector estimate $\begin{bmatrix} Q_1^k \\ Q_2^k \\ \vdots \\ Q_m^k \\ H_{\mathrm{T}}^k \end{bmatrix}$.

(c) GENERATE INITIAL ESTIMATES

The discharge Q_i may be estimated initially as being proportional to the cross-sectional area of the pipe i, i.e.,

$$Q_i^0 = \frac{A_i}{\sum_{i=1}^{m} A_i} Q_{\mathrm{T}} = \frac{{D_i}^2}{\sum_{i=1}^{m} {D_i}^2} Q_{\mathrm{T}} \qquad i = 1, 2, \ldots, m$$

Then, the total head at the pump house may be estimated as

$$H_{\mathrm{T}}^0 = \max\left[c_1 (Q_1^0)^2 + H_1, c_2 (Q_2^0)^2 + H_2, \ldots, c_m (Q_m^0)^2 + H_m\right]$$

Alternatively, H_{T} must at least be equal to the maximum elevation H_i. Hence, another set of estimates is

$$\begin{aligned} H_{\mathrm{T}}^0 &= \max(H_1, H_2, \ldots, H_m) \\ Q_i^0 &= \sqrt{\frac{H_{\mathrm{T}}^0 - H_i}{c_i}} \qquad i = 1, 2, \ldots, m \end{aligned}$$

5. Sample computations

Assume that we are given the following input data:

i	L_i, ft	D_i, in	H_i, ft	e_i, ft
1	500	6	100	0.00015
2	600	6	200	0.00015
3	1000	8	400	0.00015
4	1000	8	300	0.00015
5	800	8	500	0.00015

DETERMINE INITIAL ESTIMATES

i	$f_1 = \dfrac{1}{\left(2\log_{10}\left(\frac{D_i}{e_i}\right) + 1.14\right)^2}$	$c_i = \dfrac{8 f_i L_i}{g\pi^2 {D_i}^5}$	$Q_i^0 = \dfrac{{D_i}^2}{\sum_{i=1}^{5} {D_i}^2} Q_{\mathrm{T}}$	$c_i (Q_i^0)^2 + H_i$
1	0.01492	6.0157	6.8182	379.66
2	0.01492	7.2189	6.8182	535.59
3	0.01405	2.6885	12.1212	795.00
4	0.01405	2.6885	12.1212	695.00
5	0.01405	2.1508	12.1212	$816.00 = H_{\mathrm{T}}^0$

PERFORM ITERATIONS OF THE NEWTON-RAPHSON METHOD

1st iteration

$$\begin{aligned} \varphi_{11}^0 &= 2c_1 Q_1^0 = 2(6.0157)(6.8182) = 82.03 \\ \varphi_{22}^0 &= 2c_2 Q_2^0 = 2(7.2189)(6.8182) = 98.44 \\ \varphi_{33}^0 &= 2c_3 Q_3^0 = 2(2.6885)(12.1212) = 65.18 \\ \varphi_{44}^0 &= 2c_4 Q_4^0 = 2(2.6885)(12.1212) = 65.18 \\ \varphi_{55}^0 &= 2c_5 Q_5^0 = 2(2.1508)(12.1212) = 52.14 \end{aligned}$$

$$
\begin{aligned}
f_1^0 &= c_1(Q_1^0)^2 - H_{\mathrm{T}}^0 + H_1 = (6.0157)(6.8182)^2 - 816.00 + 100 = -436.34 \\
f_2^0 &= c_2(Q_2^0)^2 - H_{\mathrm{T}}^0 + H_2 = (7.2189)(6.8182)^2 - 816.00 + 200 = -280.41 \\
f_3^0 &= c_3(Q_3^0)^2 - H_{\mathrm{T}}^0 + H_3 = (2.6885)(12.1212)^2 - 816.00 + 400 = -21.00 \\
f_4^0 &= c_4(Q_4^0)^2 - H_{\mathrm{T}}^0 + H_4 = (2.6885)(12.1212)^2 - 816.00 + 300 = -121.00 \\
f_5^0 &= c_5(Q_5^0)^2 - H_{\mathrm{T}}^0 + H_5 = (2.1508)(12.1212)^2 - 816.00 + 500 = 0.00 \\
f_6^0 &= Q_1^0 + Q_2^0 + Q_3^0 + Q_4^0 + Q_5^0 - Q_{\mathrm{T}} = 6.8182 + 6.8182 + 12.1212 + 12.1212 + 12.1212 - 50 \\
&= 0.00
\end{aligned}
$$

$$
[\Phi_k | -\mathbf{f}_k] = \left[\begin{array}{cccccc|c}
82.03 & 0 & 0 & 0 & 0 & -1 & 436.34 \\
0 & 98.44 & 0 & 0 & 0 & -1 & 280.41 \\
0 & 0 & 65.18 & 0 & 0 & -1 & 21.00 \\
0 & 0 & 0 & 65.18 & 0 & -1 & 121.00 \\
0 & 0 & 0 & 0 & 52.14 & -1 & 0.00 \\
1 & 1 & 1 & 1 & 1 & 0 & 0.00
\end{array}\right]
$$

Using an $O(n)$ implementation of GJRM, we obtain the correction vector $\Delta\mathbf{x}_0 = \begin{bmatrix} 3.5726 \\ 1.3931 \\ -1.8761 \\ -0.3418 \\ -2.7479 \\ -143.27 \end{bmatrix}$,

from which

$$
\begin{aligned}
Q_1^1 &= 6.8182 + 3.5726 = 10.3908 \text{ cfs} \\
Q_2^1 &= 6.8182 + 1.3931 = 8.2113 \text{ cfs} \\
Q_3^1 &= 12.1212 - 1.8761 = 10.2451 \text{ cfs} \\
Q_4^1 &= 12.1212 - 0.3418 = 11.7794 \text{ cfs} \\
Q_5^1 &= 12.1212 - 2.7479 = 9.3733 \text{ cfs} \\
H_{\mathrm{T}}^1 &= 816.00 - 143.27 = 672.73 \text{ ft}
\end{aligned}
$$

with a mean relative error of

$$
\frac{1}{6}\left(\left|\frac{3.5726}{10.3908}\right| + \left|\frac{1.3931}{8.2113}\right| + \left|\frac{-1.8761}{10.2451}\right| + \left|\frac{-0.3418}{11.7794}\right| + \left|\frac{-2.7479}{9.3733}\right| + \left|\frac{-143.27}{672.73}\right|\right) = 0.2053
$$

2nd iteration (Verify.)

$$
[\Phi_k | -\mathbf{f}_k] = \left[\begin{array}{cccccc|c}
125.02 & 0 & 0 & 0 & 0 & -1 & 76.78 \\
0 & 118.55 & 0 & 0 & 0 & -1 & 14.01 \\
0 & 0 & 55.09 & 0 & 0 & -1 & 9.46 \\
0 & 0 & 0 & 63.34 & 0 & -1 & 0.31 \\
0 & 0 & 0 & 0 & 40.32 & -1 & 16.24 \\
1 & 1 & 1 & 1 & 1 & 0 & 0.00
\end{array}\right]
$$

$$
\Delta\mathbf{x}_1 = \begin{bmatrix} -0.4746 \\ 0.0290 \\ 0.1450 \\ 0.2706 \\ 0.0300 \\ 17.4504 \end{bmatrix} \qquad \begin{bmatrix} Q_1^2 \\ Q_2^2 \\ Q_3^2 \\ Q_4^2 \\ Q_5^2 \\ H_{\mathrm{T}}^2 \end{bmatrix} = \begin{bmatrix} 9.9162 \\ 8.2403 \\ 10.3901 \\ 12.0500 \\ 9.4034 \\ 690.18 \end{bmatrix} \qquad MRE = 0.0194
$$

Below is a summary of the iterations.

k	Q_1^k, cfs	Q_2^k, cfs	Q_3^k, cfs	Q_4^k, cfs	Q_5^k, cfs	H_{T}^k, ft	MRE
0	6.8182	6.8182	12.1212	12.1212	12.1212	816.00	
1	10.3908	8.2113	10.2451	11.7794	9.3733	672.73	0.2053
2	9.9162	8.2403	10.3901	12.0500	9.4034	690.18	0.0194
3	9.9066	8.2420	10.3928	12.0502	9.4084	690.38	0.0004